
ECUAȚIA DE GRADUL AL II-LEA

PROIECT TESTAREA SISTEMELOR SOFTWARE
T1 TESTARE UNITARĂ ÎN PYTHON

BILIUȚĂ Elisa-Adelina ¹¹, MARCOVICI Anamaria ²¹ și
NEDELESCU George-Eduard ³¹

¹Universitatea din București/ Facultatea de Matematică și Informatică/
Programul de studii universitare de licență Matematică-Informatică

Anul III, Seria 31, Grupa 311

¹elisa-adelina.biliuta@s.unibuc.ro

²anamaria.marcovici@s.unibuc.ro

³george-eduard.nedelescu@s.unibuc.ro

Cuprins

1	Prezentarea problemei	3
2	Testare funcțională	5
2.1	Partiționarea în clase de echivalență (CE)	5
2.2	Analiza valorilor de frontieră (AVF)	8
2.3	Partiționarea în categorii (PC)	10
2.3.1	Graful cauză-efect	13
3	Testare structurală	17
3.1	Graful de flux de control (CFG)	17
3.1.1	Generarea automată a CFG-ului	17
3.2	Analiza acoperirii (Coverage)	20
3.2.1	Acoperire la nivel de instrucțiuni	20
3.2.2	Legătura dintre cazurile de test și acoperirea codului	21
3.3	Rularea instrumentului coverage.py	21
3.4	Acoperire la nivel de condiție	22
3.4.1	Condition/Decision Coverage	23
3.4.2	Multiple Condition Coverage	23
3.4.3	MC/DC	23
3.4.4	Path Coverage	25
3.4.5	LCSAJ	25
3.5	Complexitatea ciclomatică (McCabe)	25
3.6	Circuite independente	26
4	Testare pe mutanți	28
4.1	Definiție	28
4.1.1	Principii teoretice ale mutation testing	28
4.2	Operatori de mutație utilizați	28
4.2.1	Tipuri de mutanți	28
4.3	Implementarea clasei și a mutanților	29
4.4	Teste pentru implementarea originală și pentru mutanți	33
4.4.1	Strong vs Weak Mutation	35
4.5	Analiza mutanților	35
4.5.1	Condiții pentru detectarea mutanților	35
4.6	Rezultatele rulării testelor	36
4.6.1	Mutanți echivalenți	36
4.7	Concluzii	36
4.8	Testare pe mutanți folosind Cosmic Ray	37
4.8.1	Rularea testării pe mutanți cu Cosmic Ray	38

4.8.2	Analiza mutanților supraviețuitori	39
5	Raport privind utilizarea IA pentru generarea testelor	41
5.1	Introducere	41
5.2	Promptul utilizat	41
5.3	Output-ul generat de AI	43
5.4	Clasa de teste generată	43
5.5	Tehnicile de testare acoperite	49
5.5.1	Statement coverage	49
5.5.2	Branch coverage	50
5.5.3	Condition coverage	50
5.5.4	Boundary value analysis	50
5.5.5	Equivalence partitioning	50
5.5.6	Category partitioning	50
5.5.7	Path coverage	50
5.6	Testarea output-ului în consolă	51
5.7	Mutation testing	51
5.8	Observații asupra rezultatului generat	51
5.8.1	Comparație între testele realizate manual și testele generate de AI .	52
5.9	Concluzie	54
6	Concluzii	55
6.1	Sinteza tehnicilor de testare utilizate	55
6.2	Limitări și direcții viitoare	55

Capitolul 1

Prezentarea problemei

În acest capitol prezentăm problema principală pe care o vom supune testării, anume rezolvarea unei ecuații de gradul al II-lea de forma $ax^2 + bx + c = 0$, cu $a \in \mathbb{R}^*$ și $b, c \in \mathbb{R}$.

Programul implementat primește trei coeficienți reali (a , b , și c) și calculează soluțiile ecuației pe baza discriminantului $\Delta = b^2 - 4ac$. De asemenea, programul tratează cazurile de excepție, cum ar fi validarea tipului de date introdus și restricția matematică pentru care coeficientul $a \neq 0$.

Distingem mai jos cele trei cazuri posibile, precum și formulele de calcul ale soluțiilor ecuației, în funcție de semnul lui Δ :

$$x_{1,2} = \begin{cases} \frac{-b \pm \sqrt{\Delta}}{2a}, & \text{dacă } \Delta > 0 \\ -\frac{b}{2a}, & \text{dacă } \Delta = 0 \\ \frac{-b \pm i\sqrt{|\Delta|}}{2a}, & \text{dacă } \Delta < 0 \end{cases}.$$

În continuare, prezentăm implementarea clasei `EquationSolver` în limbajul Python.

```
1 import math
2
3 class EquationSolver:
4     def __init__(self, a, b, c):
5         self.a = a
6         self.b = b
7         self.c = c
8
9     def solve(self):
10        if not all(isinstance(x, (int, float)) and not isinstance(x, bool)
11        for x in [self.a, self.b, self.c]):
12            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
13            reale!")
14
15        if self.a == 0:
16            raise ValueError("Coeficientul a nu poate fi 0!")
17
18        delta = self.b ** 2 - 4 * self.a * self.c
19
20        if math.isclose(delta, 0.0, abs_tol=1e-9):
21            return -self.b / (2 * self.a)
22
23        elif delta < 0:
24            real_part = -self.b / (2 * self.a)
25            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
26            x1 = complex(real_part, imag_part)
27            x2 = complex(real_part, -imag_part)
28            return x1, x2
29
30        else:
31            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
32            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
33            return x1, x2
```

Secvență de cod 1.1: Fișierul equation_solver.py ilustrând clasa EquationSolver

Capitolul 2

Testare funcțională

În acest capitol detaliem strategiile de testare funcțională de tip *Black Box* aplicate asupra clasei `EquationSolver`. Testarea funcțională presupune generarea datelor de test pe baza specificațiilor programului, fără a ține cont de structura internă a codului.

2.1 Partiționarea în clase de echivalență (CE)

Domeniul de intrări este format din cei trei coeficienți (a, b, c) , care trebuie să fie numere reale, iar $a \neq 0$. Pe baza verificării tipului de date introduse, a restricțiilor și a calculului discriminantului (Δ) , am identificat un domeniu de ieșiri format din 5 tipuri distincte de rezultate. În continuare, vom analiza fiecare dintre aceste domenii și clasele de echivalență generate.

Domeniul de intrări conține:

- un număr real nenul a : determină clasele $A_1 = \mathbb{R}^*$, $A_2 = \{0\}$ și $A_3 = \mathbb{R}^c$;
- un număr real b : determină clasele $B_1 = \mathbb{R}$ și $B_2 = \mathbb{R}^c$;
- un număr real c : determină clasele $C_1 = \mathbb{R}$ și $C_2 = \mathbb{R}^c$.

Observație 1. Am notat cu $\mathbb{R}^c$ complementara mulțimii numerelor reale. Dacă $x \in \mathbb{R}^c$, atunci x are tipul de dată diferit de `int` și `float` în reprezentarea internă a limbajului Python (și nu are nici tip `bool`), lucru ce va genera erori la validarea tipului de date.

Domeniul de ieșiri constă în rezultatele posibile returnate de metoda `solve()` și poate fi explicitat astfel:

- mulțimea soluțiilor ecuației: determină clasele $S_1 = \mathbb{C} \times \mathbb{C}$, $S_2 = \mathbb{R}$, $S_3 = \mathbb{R} \times \mathbb{R}$;
- stările de eroare: determină clasele: $E_1 = \text{ValueError}$ (excepție aruncată când cel puțin un coeficient aparține mulțimii $\mathbb{R}^c$) și $E_2 = \text{ValueError}$ (excepție aruncată pentru restricția matematică $a \neq 0$).

Observație 2. Clasa S_1 presupune un tuplu de două rădăcini complexe conjugate (când $\Delta < 0$), clasa S_2 presupune o singură rădăcină reală (rădăcină dublă, când $\Delta = 0$), iar clasa S_3 presupune un tuplu de două rădăcini reale distincte (când $\Delta > 0$).

Pe baza domeniului de ieșiri, folosind principiul metodei „reverse engineering” (de la ieșiri spre intrări) și ținând cont de fluxul de validare care previne evaluările multiple (o excepție aruncată oprește execuția), putem partiționa domeniul de intrări în trei clase disjuncte:

1. **Intrări care declanșează eroarea E_1 (Validarea tipului):** Execuția se oprește prematur dacă cel puțin un coeficient nu este număr real. Aceasta generează prima clasă de echivalență globală:

$$C_{E_1} = \{(a, b, c) \mid a \in A_3 \vee b \in B_2 \vee c \in C_2\}$$

2. **Intrări care nu declanșează eroarea E_1 , dar care declanșează eroarea E_2 (Restricția matematică):** Acest caz presupune că toate datele sunt numere reale ($a \in A_1 \cup A_2$, $b \in B_1$, $c \in C_1$), dar coeficientul dominant este zero ($a \in A_2$). Obținem astfel a doua clasă de echivalență globală:

$$C_{E_2} = A_2 \times B_1 \times C_1 = \{(0, b, c) \mid b, c \in \mathbb{R}\}$$

3. **Intrări care nu declanșează nicio eroare:** Acest caz presupune date valide: avem coeficienți reali și $a \neq 0$, adică domeniul $A_1 \times B_1 \times C_1$. Facem discuție după calculul discriminantului Δ , care subpartiționează domeniul valid în cele 3 clase corespunzătoare soluțiilor:

$$C_{S_1} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac < 0\}$$

$$C_{S_2} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac = 0\}$$

$$C_{S_3} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac > 0\}$$

Pe baza partiționării anterioare, am selectat câte un caz de testare reprezentativ pentru fiecare clasă de echivalență. Seturile de date și rezultatele așteptate sunt sintetizate în tabelul de mai jos:

ID Test	Clasa	a	b	c	Δ	Rezultat așteptat
TC_CE_01	C_{E_1}	"text"	2	1	N/A	E_1 (Excepție: tipuri invalide)
TC_CE_02	C_{E_2}	0	2	1	N/A	E_2 (Excepție: a nu poate fi 0)
TC_CE_03	C_{S_1}	1	1	1	-3	S_1 (Două rădăcini complexe)
TC_CE_04	C_{S_2}	1	2	1	0	S_2 (Rădăcină reală dublă)
TC_CE_05	C_{S_3}	1	5	6	1	S_3 (Două rădăcini reale distincte)

Tabela 2.1: Cazuri de testare pentru metoda CE

Observație 3. Se remarcă faptul că pentru clasele de excepție C_{E_1} și C_{E_2} , discriminantul Δ nu este calculat, deoarece execuția este întreruptă înainte de a ajunge la evaluarea formulei matematice. Această afirmație nu încalcă principiile strategiei *Black Box*.

Pentru a valida practic scenariile teoretice definite anterior, în secvența de cod următoare prezentăm implementarea automatizată a cazurilor de testare extrase din Tabelul 2.1, utilizând frameworkul `unittest` din limbajul Python. Deoarece urmează se efectuează calcule analitice pe valori reale, verificările folosesc extensiv mecanismul `assertAlmostEqual` pentru a preveni eșuarea testelor pe fondul erorilor inevitabile de trunchiere și rotunjire în virgulă mobilă.

```
1 class TestEquationSolverCE(unittest.TestCase):
2     # TC_CE_01: Clasa CE1 - Validarea tipului de date (cel puțin un
    # coeficient nu este număr real); vom trata fenomenul defect masking
3     def test_tc_ce_01_a_invalid(self):
4         solver = EquationSolver("text", 2, 1)
5         with self.assertRaises(ValueError):
6             solver.solve()
7
8     def test_tc_ce_01_b_invalid(self):
9         solver = EquationSolver(1, "text", 1)
10        with self.assertRaises(ValueError):
11            solver.solve()
12
13    def test_tc_ce_01_c_invalid(self):
14        solver = EquationSolver(1, 2, "text")
15        with self.assertRaises(ValueError):
16            solver.solve()
17
18    # TC_CE_02: Clasa CE2 - Restricția matematică ( $a \neq 0$ )
19    def test_tc_ce_02_a_zero(self):
20        solver = EquationSolver(0, 2, 1)
21        with self.assertRaises(ValueError):
22            solver.solve()
23
24    # TC_CE_03: Clasa CS1 -  $\Delta < 0$  (Două rădăcini complexe conjugate)
25    def test_tc_ce_03_delta_negativ(self):
26        solver = EquationSolver(1, 1, 1)
27        x1, x2 = solver.solve()
28
29        self.assertIsInstance(x1, complex)
30        self.assertIsInstance(x2, complex)
31
32        self.assertAlmostEqual(x1.real, -0.5)
33        self.assertAlmostEqual(x2.real, -0.5)
34
35        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
36        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)
37
38    # TC_CE_04: Clasa CS2 -  $\Delta = 0$  (0 rădăcină reală dublă)
39    def test_tc_ce_04_delta_zero(self):
40        solver = EquationSolver(1, 2, 1)
41        x = solver.solve()
42
43        self.assertIsInstance(x, float)
44        self.assertAlmostEqual(x, -1.0)
45
46    # TC_CE_05: Clasa CS3 -  $\Delta > 0$  (Două rădăcini reale distincte)
47    def test_tc_ce_05_delta_pozitiv(self):
48        solver = EquationSolver(1, 5, 6)
49        x1, x2 = solver.solve()
50
51        self.assertIsInstance(x1, float)
52        self.assertIsInstance(x2, float)
53
54        self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
55        self.assertAlmostEqual(min(x1, x2), -3.0, places=5)
```

Secvență de cod 2.1: Fișierul test_equation_solver.py ilustrând clasa TestEquationSolverCE

2.2 Analiza valorilor de frontieră (AVF)

Conform principiilor de bază ale metodei AVF, pentru fiecare frontieră identificată trebuie testată atât valoarea exactă a limitei, cât și valorile imediat adiacente (din interiorul și din afara clasei valide). Pentru problema ecuației de gradul al II-lea, deși prelucrăm un domeniu continuu, reprezentat de mulțime $\mathbb{R}$, distingem clar două frontiere decizionale impuse de fluxul de control al programului: trecerea dintre valid și invalid ($a = 0$) și punctul critic dintre rădăcinile reale și cele complexe ($\Delta = 0$). Să detaliem:

- **Frontiera coeficientului dominant (a):** Restricția matematică impune $a \neq 0$. Conform teoriei AVF, setul de teste trebuie să evalueze cazul-limită ($a = 0$) și valorile imediat adiacente acestuia ($a \rightarrow 0^+$ și $a \rightarrow 0^-$). Întrucât testul pentru valoarea $a = 0$ a fost deja executat sub umbrela metodei CE (reprezentând clasa de excepție C_{E_2}), analiza de frontieră va fi completată prin injectarea unor valori arbitrar de mici, notate cu $\pm\epsilon$ (unde vom alege $\epsilon = 0.001$).
- **Frontiera discriminantului (Δ):** Valoare critică care produce separarea rădăcinilor reale de cele complexe este $\Delta = 0$. Cazul-limită ($\Delta = 0$) reprezintă însăși clasa C_{S_2} din metoda CE. Tehnica AVF impune completarea cu testarea vecinătăților $\Delta \rightarrow 0^+$ și $\Delta \rightarrow 0^-$, stări care pot fi forțate prin variația fină a coeficientului b în jurul valorii 2, menținând $a = 1$ și $c = 1$, de exemplu.

Pentru a oferi o imagine completă a acoperirii la frontieră, Tabelul 2.2 sintetizează întregul spectru de teste AVF, înglobând atât valorile pentru frontieră (moștenite din partiționarea de echivalență), cât și cazurile pentru vecinătățile acesteia.

ID Test	Frontieră	a	b	c	Δ	Rezultat așteptat
TC_CE_02	$a = 0$	0	2	1	N/A	E_2 (Excepție: a nu poate fi 0)
TC_AVF_01	$a \rightarrow 0^-$	-0.001	2	1	4.004	S_3 (Două rădăcini reale distincte)
TC_AVF_02	$a \rightarrow 0^+$	0.001	2	1	3.996	S_3 (Două rădăcini reale distincte)
TC_CE_04	$\Delta = 0$	1	2	1	0	S_2 (Rădăcină reală dublă)
TC_AVF_03	$\Delta \rightarrow 0^-$	1	1.999	1	≈ -0.004	S_1 (Două rădăcini complexe)
TC_AVF_04	$\Delta \rightarrow 0^+$	1	2.001	1	≈ 0.004	S_3 (Două rădăcini reale distincte)

Tabela 2.2: Cazuri de testare pentru metoda AVF

Observație 4. Spre deosebire de mulțimile de tip `int`, unde valoarea adiacentă este precis determinabilă (+1 sau -1), pe un domeniu continuu de tip `float` conceptul de „valoare imediat adiacentă” este mărginit doar de limitele de reprezentare arhitecturală a mașinii. Din perspectiva strategiei *Black Box*, o toleranță mică $\epsilon = 10^{-3}$ este optimă și suficientă pentru a garanta evaluarea ramurilor decizionale la extremitățile lor critice.

Implementarea automatizată a scenariilor AVF este redată mai jos.

```

1 class TestEquationSolverAVF(unittest.TestCase):
2     # Testele pentru a=0 si Delta=0 au fost deja implementate in clasa
    TestEquationSolverCE (test_tc_ce_02 si test_tc_ce_04)
3
4     # TC_AVF_01: Limita stanga pentru coeficientul a (a -> 0-)
5     def test_tc_avf_01_a_limita_negativa(self):
6         solver = EquationSolver(-0.001, 2, 1)
7         x1, x2 = solver.solve()
8
9         self.assertIsInstance(x1, float)
10        self.assertIsInstance(x2, float)
11
12        self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
13        self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)
14
15    # TC_AVF_02: Limita dreapta pentru coeficientul a (a -> 0+)
16    def test_tc_avf_02_a_limita_pozitiva(self):
17        solver = EquationSolver(0.001, 2, 1)
18        x1, x2 = solver.solve()
19
20        self.assertIsInstance(x1, float)
21        self.assertIsInstance(x2, float)
22
23        self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
24        self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)
25
26    # TC_AVF_03: Limita stanga pentru Delta (Delta -> 0-)
27    def test_tc_avf_03_delta_limita_negativa(self):
28        solver = EquationSolver(1, 1.999, 1)
29        x1, x2 = solver.solve()
30
31        self.assertIsInstance(x1, complex)
32        self.assertIsInstance(x2, complex)
33
34        self.assertAlmostEqual(x1.real, -0.9995, places=3)
35        self.assertAlmostEqual(x2.real, -0.9995, places=3)
36
37        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
38        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)
39
40    # TC_AVF_04: Limita dreapta pentru Delta (Delta -> 0+)
41    def test_tc_avf_04_delta_limita_pozitiva(self):
42        solver = EquationSolver(1, 2.001, 1)
43        x1, x2 = solver.solve()
44
45        self.assertIsInstance(x1, float)
46        self.assertIsInstance(x2, float)
47
48        self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
49        self.assertAlmostEqual(min(x1, x2), -1.03213, places=1)

```

Secvență de cod 2.2: Fișierul test_equation_solver.py ilustrând clasa TestEquationSolverAVF

2.3 Partiționarea în categorii (PC)

Metoda partiționării în categorii extinde tehnicile anterioare, având ca scop generarea unui set de date de test care acoperă exhaustiv funcționalitatea sistemului și maximizează posibilitatea descoperirii erorilor. Această abordare structurată descompune specificația în unități testabile, identifică parametrii, categoriile acestora și partiționează fiecare categorie în alternative. Pentru a preveni explozia combinatorică inerentă și a elimina cazurile care nu au sens, se introduc constrângeri logice (condiții de selecție) asupra alternativelor.

Urmărind pașii metodologici, aplicăm metoda PC pentru clasa `EquationSolver`:

Pasul 1: Descompunerea specificației în unități

Sistemul prezintă o singură unitate funcțională testabilă izolată: metoda `solve()`.

Pasul 2: Identificarea parametrilor și a condițiilor de mediu

Parametrii de intrare care influențează direct comportamentul sunt coeficienții a, b, c . Condiția de mediu relevantă (starea internă rezultată în urma procesării datelor valide) este discriminantul Δ .

Pasul 3: Găsirea categoriilor

Proprietățile majore care definesc și separă fluxurile de execuție ale programului sunt:

- C_1 (*Validitatea tipului de date*): verifică apartenența $a, b, c \in \mathbb{R}$;
- C_2 (*Valoarea coeficientului dominant*): impune restricția $a \neq 0$;
- C_3 (*Natura rădăcinilor*): dictată de semnul lui Δ .

Pașii 4 și 5: Partiționarea în alternative și scrierea specificației de testare

Pentru a eficientiza testarea și a evita generarea de combinații redundante, asociem alternativelor constrângeri. Proprietatea `[error]` va opri combinarea ulterioară, deoarece declanșează întreruperea execuției programului. Pentru a asigura o testare robustă, alternativele integrează și valorile de frontieră identificate anterior. Să detaliem:

• Parametrul a :

1. $a \in \mathbb{R}^c$ `[error];`
2. $a = 0$ `[error];`
3. $a \rightarrow 0^-$ `[property Valid_A];`
4. $a \rightarrow 0^+$ `[property Valid_A];`
5. $a \in \mathbb{R} \setminus [-\epsilon, \epsilon]$ `[property Valid_A].`

• Parametrul b :

1. $b \in \mathbb{R}^c$ `[error];`
2. $b \in \mathbb{R}$ `[property Valid_B].`

• Parametrul c :

1. $c \in \mathbb{R}^c$ `[error];`
2. $c \in \mathbb{R}$ `[property Valid_C].`

• Semnul lui Δ :

1. $\Delta \ll 0$ `[if Valid_A and Valid_B and Valid_C];`
2. $\Delta \rightarrow 0^-$ `[if Valid_A and Valid_B and Valid_C];`
3. $\Delta = 0$ `[if Valid_A and Valid_B and Valid_C];`
4. $\Delta \rightarrow 0^+$ `[if Valid_A and Valid_B and Valid_C];`
5. $\Delta \gg 0$ `[if Valid_A and Valid_B and Valid_C].`

Pașii 6 și 7: Crearea cazurilor și a datelor de test

În absența constrângerilor, simplul produs cartezian al alternativelor ar fi generat $5 \times 2 \times 2 \times 5 = 100$ de combinații. Aplicând condițiile de selecție, excludem evaluarea lui Δ pentru intrările care declanșează erori. De asemenea, pentru optimizare, într-un caz de testare la limita frontierelor, testăm doar câte o singură frontieră, reducând astfel spațiul teoretic la 11 scenarii de testare distincte, care pot fi observate în Tabelul 2.3.

Acest set de teste este optim, deoarece cuprinde:

- **4 cazuri de excepție:** acoperă validarea datelor (3 teste) și restricția matematică esențială (1 test);
- **2 cazuri pentru frontiera intrărilor:** testează stabilitatea programului când a tinde la 0 din ambele direcții;
- **2 cazuri pentru frontiera stărilor interne:** testează sensibilitatea decizională la granița dintre soluțiile reale și cele complexe;
- **3 cazuri de clasă normală:** validează comportamentul standard pentru cele trei tipuri de rădăcini.

ID Test	Combinații	a	b	c	Rezultat așteptat
TC_PC_01	a_1	True	2	1	E_1 (Excepție: tipuri invalide)
TC_PC_02	$a_5 \wedge b_1$	1	None	1	E_1 (Excepție: tipuri invalide)
TC_PC_03	$a_5 \wedge b_2 \wedge c_1$	1	2	[]	E_1 (Excepție: tipuri invalide)
TC_PC_04	$a_2 \wedge b_2 \wedge c_2$	0	2	1	E_2 (Excepție: a nu poate fi 0)
TC_PC_05	$a_3 \wedge b_2 \wedge c_2 \wedge \Delta_5$	-0.001	2	1	S_3 ($a \rightarrow 0^-$; $\Delta > 0$)
TC_PC_06	$a_4 \wedge b_2 \wedge c_2 \wedge \Delta_5$	0.001	2	1	S_3 ($a \rightarrow 0^+$; $\Delta > 0$)
TC_PC_07	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_2$	1	1.999	1	S_1 (a normal; $\Delta \rightarrow 0^-$)
TC_PC_08	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_3$	1	2	1	S_2 (a normal; $\Delta = 0$)
TC_PC_09	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_4$	1	2.001	1	S_3 (a normal; $\Delta \rightarrow 0^+$)
TC_PC_10	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_1$	1	1	1	S_1 (valori normale; $\Delta < 0$)
TC_PC_11	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_5$	1	5	6	S_3 (valori normale; $\Delta > 0$)

Tabela 2.3: Cazuri de testare pentru metoda PC

Pentru a încheia secțiunea, prezentăm implementarea automatizată a scenariilor summarize mai sus în fragmentul de cod următor.

```

1 class TestEquationSolverPC(unittest.TestCase):
2     # TC_PC_01 -> TC_PC_03: Erori de validare a tipului de date pentru
    coeficientii a, b, c
3     def test_tc_pc_01_a_invalid(self):
4         solver = EquationSolver(True, 2, 1)
5         with self.assertRaises(ValueError):
6             solver.solve()
7
8     def test_tc_pc_02_b_invalid(self):
9         solver = EquationSolver(1, None, 1)
10        with self.assertRaises(ValueError):
11            solver.solve()
12

```

```

13 def test_tc_pc_03_c_invalid(self):
14     solver = EquationSolver(1, 2, [])
15     with self.assertRaises(ValueError):
16         solver.solve()
17
18 # TC_PC_04: Valoare invalida matematic (a = 0)
19 def test_tc_pc_04_a_zero(self):
20     solver = EquationSolver(0, 2, 1)
21     with self.assertRaises(ValueError):
22         solver.solve()
23
24 # TC_PC_05: Limita stanga pentru coeficientul a (a -> 0-)
25 def test_tc_pc_05_a_limita_negativa(self):
26     solver = EquationSolver(-0.001, 2, 1)
27     x1, x2 = solver.solve()
28
29     self.assertIsInstance(x1, float)
30     self.assertIsInstance(x2, float)
31
32     self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
33     self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)
34
35 # TC_PC_06: Limita dreapta pentru coeficientul a (a -> 0+)
36 def test_tc_pc_06_a_limita_pozitiva(self):
37     solver = EquationSolver(0.001, 2, 1)
38     x1, x2 = solver.solve()
39
40     self.assertIsInstance(x1, float)
41     self.assertIsInstance(x2, float)
42
43     self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
44     self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)
45
46 # TC_PC_07: Limita stanga pentru Delta (Delta -> 0-)
47 def test_tc_pc_07_delta_limita_negativa(self):
48     solver = EquationSolver(1, 1.999, 1)
49     x1, x2 = solver.solve()
50
51     self.assertIsInstance(x1, complex)
52     self.assertIsInstance(x2, complex)
53
54     self.assertAlmostEqual(x1.real, -0.9995, places=3)
55     self.assertAlmostEqual(x2.real, -0.9995, places=3)
56
57     self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
58     self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)
59
60 # TC_PC_08: Delta = 0 (0 radacina reala dubla)
61 def test_tc_pc_08_delta_zero(self):
62     solver = EquationSolver(1, 2, 1)
63     x = solver.solve()
64
65     self.assertIsInstance(x, float)
66     self.assertEqual(x, -1.0)
67
68 # TC_PC_09: Limita dreapta pentru Delta (Delta -> 0+)
69 def test_tc_pc_09_delta_limita_pozitiva(self):
70     solver = EquationSolver(1, 2.001, 1)

```

```

71     x1, x2 = solver.solve()
72
73     self.assertIsInstance(x1, float)
74     self.assertIsInstance(x2, float)
75
76     self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
77     self.assertAlmostEqual(min(x1, x2), -1.03213, places=3)
78
79     # TC_PC_10: Delta < 0 (Doua radacini complexe)
80     def test_tc_pc_10_delta_negativ(self):
81         solver = EquationSolver(1, 1, 1)
82         x1, x2 = solver.solve()
83
84         self.assertIsInstance(x1, complex)
85         self.assertIsInstance(x2, complex)
86
87         self.assertAlmostEqual(x1.real, -0.5)
88         self.assertAlmostEqual(x2.real, -0.5)
89
90         self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
91         self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)
92
93     # TC_PC_11: Delta > 0 (Doua radacini reale distincte)
94     def test_tc_pc_11_delta_pozitiv(self):
95         solver = EquationSolver(1, 5, 6)
96         x1, x2 = solver.solve()
97
98         self.assertIsInstance(x1, float)
99         self.assertIsInstance(x2, float)
100
101         self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
102         self.assertAlmostEqual(min(x1, x2), -3.0, places=5)

```

Secvență de cod 2.3: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverPC`

Rezultatul obținut la rularea tuturor testelor din acest capitol este prezentat mai jos:

```

"D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\venv\Scripts\
python.exe" "D:/SOFTURI/PyCharm 2025.3.3/plugins/python-ce/helpers/
pycharm/_jb_unittest_runner.py" --path "D:\FMI\ANUL III\TESTAREA
SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py"
Testing started at 17:02 ...
Launching unittests with arguments python -m unittest D:\FMI\ANUL III\
TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py --quiet
in D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS

Ran 22 tests in 0.024s

OK

Process finished with exit code 0

```

2.3.1 Graful cauză-efect

Pentru a evidenția relația dintre condițiile de intrare și rezultatele posibile ale programului, putem defini un set de cauze și efecte asociate metodei `solve()`.

Cauzele identificate sunt:

- C1: cel puțin un coeficient nu este numeric;
- C2: coeficientul a este egal cu 0;
- C3: discriminantul este negativ, $\Delta < 0$;
- C4: discriminantul este egal cu 0, $\Delta = 0$;
- C5: discriminantul este pozitiv, $\Delta > 0$.

Efectele corespunzătoare sunt:

- E1: se generează `ValueError` pentru tipuri invalide;
- E2: se generează `ValueError` pentru $a = 0$;
- E3: se returnează două rădăcini complexe conjugate;
- E4: se returnează o rădăcină reală dublă;
- E5: se returnează două rădăcini reale distincte.

Relațiile cauză-efect pot fi sintetizate astfel:

$$C1 \Rightarrow E1$$

$$\neg C1 \wedge C2 \Rightarrow E2$$

$$\neg C1 \wedge \neg C2 \wedge C3 \Rightarrow E3$$

$$\neg C1 \wedge \neg C2 \wedge C4 \Rightarrow E4$$

$$\neg C1 \wedge \neg C2 \wedge C5 \Rightarrow E5$$

Această analiză confirmă faptul că fiecare rezultat posibil al metodei este determinat de o combinație clară de condiții de intrare și condiții interne.

Pentru a transforma analiza cauză-efect într-o formă mai apropiată de metodologia prezentată la curs, construim și un tabel de decizie. Acesta arată explicit ce combinații de cauze conduc la fiecare efect observabil al metodei `solve()`.

Notăm cu T faptul că o cauză este adevărată, cu F faptul că este falsă, iar cu “–” faptul că valoarea cauzei nu mai este relevantă, deoarece execuția programului s-a oprit anterior printr-o excepție.

Cauză / Efect	R1	R2	R3	R4	R5
C_1 : coeficient invalid	T	F	F	F	F
C_2 : $a = 0$	–	T	F	F	F
C_3 : $\Delta < 0$	–	–	T	F	F
C_4 : $\Delta = 0$	–	–	F	T	F
C_5 : $\Delta > 0$	–	–	F	F	T
E_1 : <code>ValueError</code> pentru tip invalid	X				
E_2 : <code>ValueError</code> pentru $a = 0$		X			
E_3 : două rădăcini complexe			X		
E_4 : rădăcină reală dublă				X	
E_5 : două rădăcini reale distincte					X

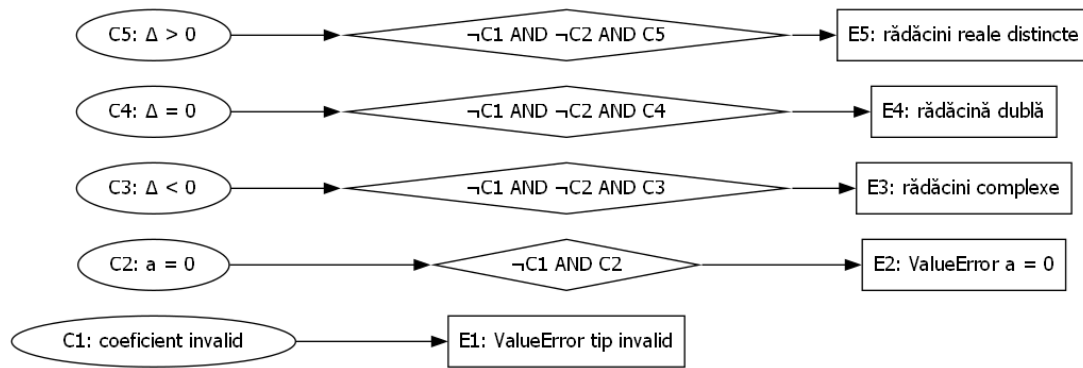
Tabela 2.4: Tabel de decizie pentru graful cauză-efect al metodei `solve()`

Regulile din tabel corespund celor cinci drumuri funcționale principale ale metodei: validare eșuată a tipului de date, încălcarea restricției $a \neq 0$, discriminant negativ, discriminant nul și discriminant pozitiv. Astfel, tabelul de decizie confirmă faptul că fiecare efect observabil este produs de o combinație unică și neambiguă de cauze.

```

1 from graphviz import Digraph
2
3 g = Digraph("CauseEffectGraph", format="png")
4 g.attr(rankdir="LR")
5
6 # Cauze
7 g.node("C1", "C1: coeficient invalid", shape="ellipse")
8 g.node("C2", "C2: a = 0", shape="ellipse")
9 g.node("C3", "C3: delta < 0", shape="ellipse")
10 g.node("C4", "C4: delta = 0", shape="ellipse")
11 g.node("C5", "C5: delta > 0", shape="ellipse")
12
13 # Noduri logice
14 g.node("N1", "!C1 && C2", shape="diamond")
15 g.node("N2", "!C1 && !C2 && C3", shape="diamond")
16 g.node("N3", "!C1 && !C2 && C4", shape="diamond")
17 g.node("N4", "!C1 && !C2 && C5", shape="diamond")
18
19 # Efecte
20 g.node("E1", "E1: ValueError tip invalid", shape="box")
21 g.node("E2", "E2: ValueError a = 0", shape="box")
22 g.node("E3", "E3: radacini complexe", shape="box")
23 g.node("E4", "E4: radacina dubla", shape="box")
24 g.node("E5", "E5: radacini reale distincte", shape="box")
25
26 # Legaturi
27 g.edge("C1", "E1")
28
29 g.edge("C2", "N1")
30 g.edge("N1", "E2")
31
32 g.edge("C3", "N2")
33 g.edge("N2", "E3")
34
35 g.edge("C4", "N3")
36 g.edge("N3", "E4")
37
38 g.edge("C5", "N4")
39 g.edge("N4", "E5")
40
41 g.render("cause_effect_graph", view=True)

```


Figura 2.1: Graful cauză-efect pentru metoda `solve()`

Capitolul 3

Testare structurală

Testarea structurală (White-Box Testing) reprezintă o metodă de testare care analizează structura internă a codului sursă. Aceasta urmărește fluxul de control, ramificațiile și condițiile logice ale programului, având ca scop executarea tuturor instrucțiunilor și deciziilor cel puțin o dată în timpul testării.

În cadrul acestui capitol se realizează analiza structurală a metodei `solve()` din clasa `EquationSolver`, prin construirea grafului de flux de control (Control Flow Graph), determinarea complexității ciclomatice, identificarea drumurilor independente și evaluarea gradului de acoperire a codului pe baza cazurilor de test definite anterior.

3.1 Graful de flux de control (CFG)

Graful de flux de control (CFG) reprezintă o modelare grafică a modului în care instrucțiunile unui program sunt executate. Nodurile grafului corespund blocurilor de instrucțiuni, iar muchiile indică tranzițiile dintre acestea.

Analiza CFG permite evidențierea ramificațiilor logice și identificarea drumurilor de execuție ale programului.

Pentru metoda `solve()`, graful de flux de control surprinde etapele principale ale algoritmului: validarea coeficienților, verificarea condiției $a = 0$, calculul discriminantului și ramificarea execuției în funcție de valoarea acestuia.

3.1.1 Generarea automată a CFG-ului

Pentru automatizarea procesului de construire a grafului de flux de control, a fost utilizată biblioteca `Graphviz`. Codul de mai jos definește nodurile și muchiile corespunzătoare fluxului de execuție al metodei `solve()`.

```
1 from graphviz import Digraph
2
3 cfg = Digraph("CFG")
4
5 # Noduri
6 cfg.node("1", "Start")
7 cfg.node("2", "Intrari valide?")
8 cfg.node("3", "ValueError")
9 cfg.node("4", "a == 0?")
10 cfg.node("5", "ValueError")
11 cfg.node("6", " $\delta = b^2 - 4ac$ ")
12 cfg.node("7", " $\delta < 0?$ ")
13 cfg.node("8", "Radacini complexe")
14 cfg.node("9", " $\delta = 0?$ ")
15 cfg.node("10", "Radacina dubla")
16 cfg.node("11", "Doua radacini reale")
17 cfg.node("12", "Stop")
18
19 # Muchii
20 cfg.edge("1", "2")
21
22 cfg.edge("2", "3", label="Nu")
23 cfg.edge("2", "4", label="Da")
24 cfg.edge("3", "12")
25
26 cfg.edge("4", "5", label="Da")
27 cfg.edge("4", "6", label="Nu")
28 cfg.edge("5", "12")
29
30 cfg.edge("6", "7")
31
32 cfg.edge("7", "8", label="Da")
33 cfg.edge("7", "9", label="Nu")
34 cfg.edge("8", "12")
35
36 cfg.edge("9", "10", label="Da")
37 cfg.edge("9", "11", label="Nu")
38
39 cfg.edge("10", "12")
40 cfg.edge("11", "12")
41
42 # Genereaza imaginea
43 cfg.render("equation_solver_cfg", format="png", cleanup=True)
44
45 print("CFG generated successfully!")
```

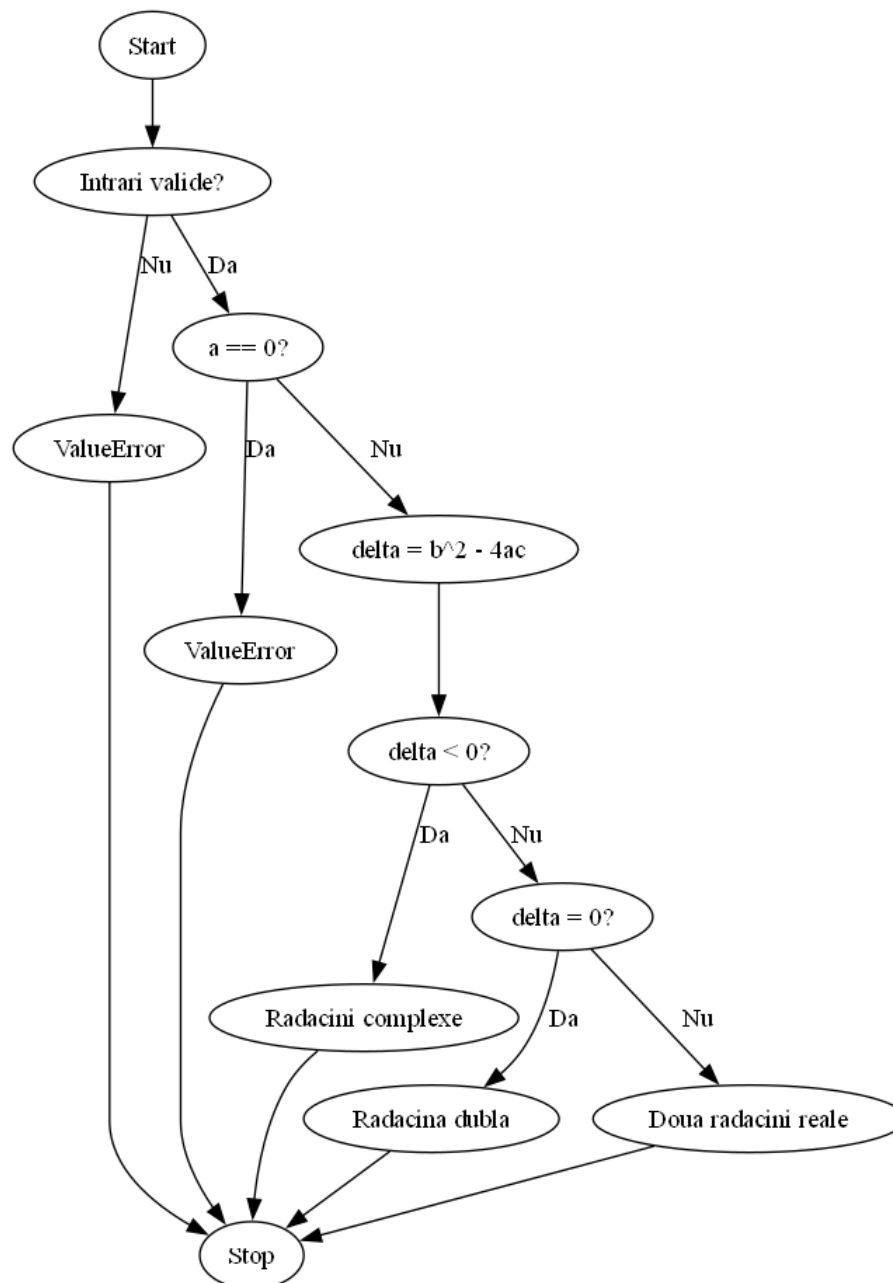


Figura 3.1: Graful de flux de control generat automat folosind Graphviz pentru metoda `solve()`

Graful evidențiază succesiunea pașilor de execuție. Execuția începe în nodul *Start*, unde sunt validate intrările. Dacă cel puțin un coeficient nu este numeric, execuția este întreruptă prin generarea unei excepții.

În cazul în care datele sunt valide, se verifică valoarea coeficientului a . Dacă $a = 0$, execuția este oprită, deoarece ecuația nu mai este de gradul al doilea. În caz contrar, se calculează discriminantul $\Delta = b^2 - 4ac$.

În funcție de valoarea discriminantului, execuția se ramifică în trei situații:

- $\Delta < 0$ – două rădăcini complexe;

- $\Delta = 0$ – o rădăcină reală dublă;
- $\Delta > 0$ – două rădăcini reale distincte.

După determinarea rezultatului, execuția ajunge în nodul final *Stop*.

3.2 Analiza acoperirii (Coverage)

Analiza acoperirii codului urmărește în ce măsură cazurile de test exercită diferitele componente ale programului. Scopul este verificarea faptului că toate instrucțiunile și toate ramurile logice sunt executate cel puțin o dată.

Acoperirea la nivel de decizie este strâns corelată cu acoperirea la nivel de ramură (branch coverage), deoarece fiecare decizie produce ramuri distincte de execuție. Pentru metoda `solve()`, sunt analizate două tipuri principale de acoperire: acoperirea la nivel de instrucțiuni și acoperirea la nivel de decizie.

3.2.1 Acoperire la nivel de instrucțiuni

Acoperirea la nivel de instrucțiuni (Statement Coverage) presupune executarea fiecărei instrucțiuni cel puțin o dată.

În cazul metodei `solve()`, acest lucru implică testarea tuturor situațiilor posibile:

- intrări invalide (coeficienți nenumerici);
- cazul invalid în care $a = 0$;
- $\Delta < 0$;
- $\Delta = 0$;
- $\Delta > 0$.

Test Case	Input	Rezultat	Instrucțiuni/Noduri parcurse
TC_CE_01	(a="x", b=2, c=1)	ValueError	Start → validare input → eroare
TC_CE_02	(a=0, b=2, c=1)	ValueError	Start → validare → $a = 0$ → eroare
TC_CE_03	(1,1,1)	rădăcini complexe	Start → $\Delta < 0$
TC_CE_04	(1,2,1)	rădăcină dublă	Start → $\Delta = 0$
TC_CE_05	(1,5,6)	două rădăcini reale	Start → $\Delta > 0$

Tabela 3.1: Statement coverage pentru metoda `solve()`

Acoperirea la nivel de decizie presupune evaluarea fiecărei condiții logice atât cu valoarea *true*, cât și *false*.

Pentru metoda analizată, aceasta implică:

- verificarea validității intrărilor;

- verificarea condiției $a = 0$;
- evaluarea celor trei cazuri determinate de discriminant.

Prin cazurile de test definite în Capitolul 2, toate ramurile logice sunt exercitate, asigurând acoperire completă la nivel de instrucțiuni și decizie.

Decizie	True	False
Intrări valide?	TC_CE_02, TC_CE_03, TC_CE_04, TC_CE_05	TC_CE_01
$a == 0$	TC_CE_02	TC_CE_03, TC_CE_04, TC_CE_05
$\Delta < 0$	TC_CE_03	TC_CE_04, TC_CE_05
$\Delta = 0$	TC_CE_04	TC_CE_03, TC_CE_05

Tabela 3.2: Decision/Branch coverage pentru metoda solve()

3.2.2 Legătura dintre cazurile de test și acoperirea codului

Pentru a demonstra acoperirea completă, corelăm cazurile de test definite cu ramurile din CFG.

- **Validarea tipului de date (E_1)**
acoperită de: TC_CE_01, TC_PC_01, TC_PC_02, TC_PC_03
- **Restricția $a = 0$ (E_2)**
acoperită de: TC_CE_02, TC_PC_04
- $\Delta < 0$
acoperit de: TC_CE_03, TC_PC_10, TC_AVF_03
- $\Delta = 0$
acoperit de: TC_CE_04, TC_PC_08
- $\Delta > 0$
acoperit de: TC_CE_05, TC_PC_11, TC_AVF_01, TC_AVF_02, TC_AVF_04

3.3 Rularea instrumentului coverage.py

Pentru a valida practic gradul de acoperire al codului, am utilizat instrumentul `coverage.py`. Acesta permite monitorizarea liniilor de cod executate în timpul rulării testelor automate și generează un raport care evidențiază numărul de instrucțiuni executate, instrucțiunile neacoperite și procentul total de acoperire.

Înainte de rularea testelor, am șters rezultatele anterioare de coverage folosind comanda `coverage erase`. Ulterior, testele au fost executate prin framework-ul `unittest`, iar raportul final a fost generat cu opțiunea `-m`, care afișează și liniile lipsă, dacă acestea există.

Comanda utilizată a fost:

```
coverage erase
coverage run --branch -m unittest test_equation_solver.py
coverage report -m
```

Rezultatul obținut în consolă a fost următorul:

```
Ran 22 tests in 0.007s
```

```
OK
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
-----	-----	-----	-----	-----	-----	-----
equation_solver.py	23	0	8	0	100%	
test_equation_solver.py	139	0	0	0	100%	
-----	-----	-----	-----	-----	-----	-----
TOTAL	162	0	8	0	100%	

După cum se observă, fișierul *equation_solver.py* are o acoperire de **100%**, ceea ce înseamnă că toate instrucțiunile din implementarea clasei `Equation Solver` au fost executate cel puțin o dată de cazurile de test definite.

De asemenea, fișierul *test_equation_solver.py* are tot o acoperire de **100%**, iar raportul total indică 162 de instrucțiuni executate și 0 instrucțiuni neacoperite. Acest rezultat confirmă faptul că setul de teste propus acoperă complet implementarea analizată, incluzând atât cazurile de succes, cât și cazurile de eroare. Prin această corespondență se confirmă că toate ramurile logice sunt testate, iar acoperirea codului este completă.

3.4 Acoperire la nivel de condiție

Acoperirea la nivel de condiție (Condition Coverage) urmărește evaluarea fiecărei condiții elementare dintr-o expresie logică atât cu valoarea adevărat, cât și fals.

Scopul acestei metrice este de a verifica toate componentele unei condiții, nu doar rezultatul final al deciziei.

În cazul metodei `solve()`, condițiile sunt în mare parte simple (nu există expresii logice compuse explicite), astfel încât acoperirea la nivel de condiție coincide în mare măsură cu acoperirea la nivel de decizie.

Condițiile identificate în cod sunt:

- validarea tipului coeficienților: `not all(...)`
- verificarea coeficientului dominant: `a == 0`
- evaluarea discriminantului: `delta < 0`
- verificarea cazului limită: `math.isclose(delta, 0)`

Pentru a demonstra acoperirea la nivel de condiție, analizăm pentru fiecare condiție cazurile în care aceasta este evaluată atât ca adevărat, cât și fals.

Condiție	Evaluare True	Evaluare False
<code>not all(...)</code>	TC_CE_01	TC_CE_02–05, TC_PC_04–11, TC_AVF_01–04
<code>a == 0</code>	TC_CE_02, TC_PC_04	toate testele valide cu $a \neq 0$
<code>delta < 0</code>	TC_CE_03, TC_PC_10, TC_AVF_03	TC_CE_04–05, TC_PC_08–09, TC_AVF_01–02–04
<code>isclose(delta, 0)</code>	TC_CE_04, TC_PC_08	cazurile cu $\Delta < 0$ și $\Delta > 0$

Tabela 3.3: Acoperirea condițiilor pentru metoda `solve()`

Se observă că fiecare condiție este evaluată atât cu valoarea adevărat, cât și cu valoarea fals, ceea ce confirmă obținerea unei acoperiri complete la nivel de condiție.

3.4.1 Condition/Decision Coverage

Acoperirea condiție/decizie (Condition/Decision Coverage) combină acoperirea la nivel de condiție cu acoperirea la nivel de decizie, impunând atât evaluarea condițiilor individuale, cât și a rezultatului global al deciziei.

Această metrică oferă o analiză mai riguroasă decât acoperirea la nivel de decizie simplă.

Pentru metoda `solve()`, această acoperire este satisfăcută, deoarece fiecare condiție este evaluată pentru ambele rezultate posibile, iar fiecare ramură decizională este parcursă de cel puțin un caz de test.

3.4.2 Multiple Condition Coverage

Acoperirea la nivel de condiții multiple (Multiple Condition Coverage) presupune testarea tuturor combinațiilor posibile de valori de adevăr ale condițiilor individuale dintr-o expresie logică.

Această metodă este mai strictă decât acoperirea condiție/decizie, însă poate conduce la un număr foarte mare de cazuri de test.

În cazul metodei `solve()`, nu există condiții compuse explicite (de tip `c1 && c2` sau `c1 || c2`), astfel încât această metrică nu aduce informații suplimentare relevante.

3.4.3 MC/DC

Acoperirea MC/DC (Modified Condition/Decision Coverage) impune ca fiecare condiție individuală dintr-o decizie să ia atât valoarea **True**, cât și valoarea **False**, ca fiecare decizie să fie evaluată atât cu rezultat adevărat, cât și cu rezultat fals, și ca fiecare condiție individuală să influențeze în mod independent rezultatul deciziei.

Această metodă este mai puternică decât acoperirea condiție/decizie simplă, deoarece nu verifică doar evaluarea condițiilor, ci și efectul fiecărei condiții asupra rezultatului global al deciziei. În același timp, MC/DC evită explozia combinatorică specifică acoperirii condițiilor multiple.

Pentru metoda analizată, MC/DC este relevantă în special pentru prima decizie din metoda `solve()`, unde se realizează validarea coeficienților:

```
if not all(isinstance(x, (int, float)) and not isinstance(x, bool)
           for x in [self.a, self.b, self.c])
```

Această decizie conține o condiție compusă. Pentru fiecare coeficient x , validarea poate fi exprimată astfel:

$$V(x) = C_1(x) \wedge C_2(x)$$

unde:

$$C_1(x) = \text{isinstance}(x, (\text{int}, \text{float}))$$

și

$$C_2(x) = \text{not isinstance}(x, \text{bool})$$

Astfel, condiția C_1 verifică dacă valoarea este de tip numeric, iar condiția C_2 verifică dacă valoarea nu este de tip boolean. Această a doua verificare este necesară deoarece, în Python, tipul `bool` este subclasă a tipului `int`, iar valorile `True` și `False` ar fi acceptate de o verificare simplă cu `isinstance(x, (int, float))`.

Pentru a evidenția principiul MC/DC, putem analiza validarea unui singur coeficient, păstrând ceilalți coeficienți valizi. De exemplu:

$$V(x) = C_1 \wedge C_2$$

Test	Valoare x	C_1	C_2	$V(x)$
t1	2	True	True	True
t2	"text"	False	True	False
t3	True	True	False	False

Perechea **t1–t2** demonstrează influența independentă a condiției C_1 , deoarece C_2 rămâne adevărată, iar rezultatul validării se schimbă din adevărat în fals. Perechea **t1–t3** demonstrează influența independentă a condiției C_2 , deoarece C_1 rămâne adevărată, iar rezultatul validării se schimbă din adevărat în fals.

Prin urmare, pentru prima decizie din metoda `solve()`, MC/DC poate fi aplicată și este acoperită prin teste care includ coeficienți numerici valizi, coeficienți nenumerici și valori de tip boolean. Pentru celelalte decizii din metodă, precum `self.a == 0`, `math.isclose(delta, 0.0, abs_tol=1e-9)` și `delta < 0`, condițiile sunt simple, astfel încât MC/DC nu aduce informații suplimentare semnificative față de acoperirea la nivel de decizie.

3.4.4 Path Coverage

Testarea la nivel de cale (Path Coverage) presupune generarea unor cazuri de test astfel încât fiecare cale posibilă din graful de flux de control să fie parcursă.

În cazul metodei `solve()`, căile de execuție sunt finite și corespund ramurilor principale:

- eroare de tip (E_1)
- eroare pentru $a = 0$ (E_2)
- $\Delta < 0$
- $\Delta = 0$
- $\Delta > 0$

Acestea sunt deja acoperite prin cazurile de test definite în Capitolul 2, ceea ce asigură o acoperire completă la nivel de cale.

3.4.5 LCSAJ

Acoperirea LCSAJ (Linear Code Sequence and Jump) analizează secvențele liniare de cod urmate de salturi ale fluxului de execuție.

Această metodă oferă o granularitate mai mare decât acoperirea la nivel de decizie.

Pentru metoda `solve()`, structura simplă a codului și numărul redus de ramuri nu justifică aplicarea acestei metode, deoarece nu aduce beneficii suplimentare semnificative.

3.5 Complexitatea ciclomatică (McCabe)

Complexitatea ciclomatică reprezintă o măsură a complexității logice a unui program și indică numărul minim de drumuri independente care trebuie testate pentru a asigura acoperirea completă a codului. Aceasta a fost introdusă de McCabe și se calculează folosind relația:

$$V(G) = E - N + 2P$$

unde:

- E reprezintă numărul de muchii ale grafului de flux de control;
- N reprezintă numărul de noduri ale grafului;
- P reprezintă numărul de componente conexe ale grafului (în cazul analizat, $P = 1$).

Complexitatea ciclomatică poate fi determinată și prin numărul de decizii din program, folosind relația:

$$V(G) = D + 1$$

unde D reprezintă numărul de decizii.

În cazul metodei `solve()`, se identifică patru decizii principale:

- verificarea validității coeficienților;
- verificarea condiției $a = 0$;
- verificarea condiției $\Delta < 0$;
- verificarea condiției $\Delta = 0$ (`math.isclose(delta, 0)`).

Astfel, complexitatea ciclomatică este:

$$V(G) = 4 + 1 = 5$$

Această valoare indică faptul că există cinci drumuri independente care trebuie testate pentru a asigura acoperirea completă a metodei.

Alternativ, folosind formula bazată pe graful de control:

$$V(G) = E - N + 2P$$

unde:

- $E = 15$ (muchii)
- $N = 12$ (noduri)
- $P = 1$ (componentele conexe)

$$V(G) = 15 - 12 + 2 = 5$$

Analiza realizată se bazează pe tehnica *Basis Path Testing*, care utilizează complexitatea ciclomatică pentru a determina setul minim de drumuri independente necesare testării complete.

3.6 Circuite independente

Circuitele independente (sau drumurile independente) reprezintă trasee distincte în graful de flux de control, fiecare introducând cel puțin o muchie nouă față de drumurile parcurse anterior

Numărul acestora este determinat de complexitatea ciclomatică. În cazul metodei `solve()`, complexitatea ciclomatică este:

$$V(G) = 5$$

Prin urmare, există cinci drumuri independente care trebuie testate.

- **Drumul 1:** validarea intrărilor eșuează, iar metoda generează `ValueError` pentru coeficienți nenumerici;
- **Drumul 2:** coeficienții sunt validați cu succes, însă $a = 0$, iar metoda generează `ValueError`;
- **Drumul 3:** coeficienții sunt validați, $a \neq 0$, iar $\Delta < 0$, rezultând două rădăcini complexe;
- **Drumul 4:** coeficienții sunt validați, $a \neq 0$, iar $\Delta = 0$, rezultând o rădăcină dublă;

- **Drumul 5:** coeficienții sunt validați, $a \neq 0$, iar $\Delta > 0$, rezultând două rădăcini reale distincte.

Testarea acestor cinci drumuri asigură parcurgerea tuturor ramurilor logice din cadrul metodei `solve()`. Rezultatele obținute confirmă faptul că metoda analizată are o structură logică relativ simplă, ceea ce facilitează procesul de testare, analiză și mentenanță ulterioară.

Drum	Cale în CFG
P1	$1 \rightarrow 2 \rightarrow 3 \rightarrow 12$
P2	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 12$
P3	$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 12$
P4	$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 10 \rightarrow 12$
P5	$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 11 \rightarrow 12$

Tabela 3.4: Drumurile independente identificate în CFG

Capitolul 4

Testare pe mutanți

4.1 Definiție

Testarea pe mutanți (Mutation Testing) este o tehnică de testare structurală utilizată pentru a evalua calitatea și eficiența unui set de teste. Ideea principală constă în introducerea unor modificări mici și controlate în codul sursă, numite *mutanți*, care simulează erori de programare plauzibile.

După generarea mutanților, se rulează setul de teste existent asupra fiecărei variante modificate a programului. Dacă testele detectează modificarea și eșuează, mutantul este considerat *ucis* (*killed*). Dacă testele continuă să treacă, mutantul *supraviețuiește* (*survives*), ceea ce indică o slăbiciune a setului de teste.

Prin urmare, testarea pe mutanți oferă o măsură mai riguroasă a calității testelor decât simpla analiză de acoperire, deoarece verifică nu doar dacă o porțiune de cod a fost executată, ci și dacă testele sunt suficient de precise pentru a detecta comportamente incorecte.

4.1.1 Principii teoretice ale mutation testing

Mutation testing se bazează pe două ipoteze fundamentale:

- **Competent Programmer Hypothesis (CPH)**: se presupune că un programator scrie un program apropiat de unul corect, iar erorile sunt mici deviații.
- **Coupling Effect**: testele care detectează erori simple (mutanți de ordin 1) sunt, în general, suficiente pentru a detecta și erori mai complexe.

4.2 Operatori de mutație utilizați

Pentru evaluarea clasei `EquationSolver`, au fost selectate mai multe categorii de operatori de mutație, care vizează punctele critice ale algoritmului: calculul discriminantului, condițiile de ramificare, validarea datelor de intrare și valorile returnate de metodă.

4.2.1 Tipuri de mutanți

Mutanții pot fi clasificați în funcție de numărul de modificări aplicate:

- **Mutanți de ordin 1 (first-order mutants)**: obținuți printr-o singură modificare.

- **Mutanți de ordin superior (higher-order mutants):** obținuți prin aplicarea mai multor modificări succesive.

În practică, se utilizează în principal mutanții de ordin 1, deoarece numărul mutanților crește foarte rapid pentru ordine mai mari.

Tabela 4.1: Clasificarea mutanților pentru metoda `solve()`

Tip mutant	Cod original	Cod mutant	Explicație
AOR	<code>b**2 - 4*a*c</code>	<code>b**2 + 4*a*c</code>	Modifică formula de calcul a discriminantului.
ROR	<code>delta < 0</code>	<code>delta <= 0</code>	Afectează decizia de la frontiera $\Delta = 0$.
LCR	<code>all(...)</code>	<code>any(...)</code>	Slăbește validarea logică a coeficienților introduși.
STD-1	<code>return (x1, x2)</code>	<code>return None</code>	Modifică rezultatul returnat pe ramura cu rădăcini complexe.
STD-2	<code>return -b/(2*a)</code>	<code>return -b/a</code>	Modifică formula pentru cazul $\Delta = 0$.

4.3 Implementarea clasei și a mutanților

În continuare este prezentată implementarea clasei originale `EquationSolver`, precum și a mutanților utilizați în analiză.

```

1 import math
2 import unittest
3
4
5 class EquationSolver:
6     def __init__(self, a, b, c):
7         self.a = a
8         self.b = b
9         self.c = c
10
11     def solve(self):
12         if not all(
13             isinstance(x, (int, float)) and not isinstance(x, bool)
14             for x in [self.a, self.b, self.c]
15         ):
16             raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
17 reale!")
18
19         if self.a == 0:
20             raise ValueError("Coeficientul a nu poate fi 0!")
21
22         delta = self.b ** 2 - 4 * self.a * self.c
23
24         if math.isclose(delta, 0.0, abs_tol=1e-9):
25             return -self.b / (2 * self.a)
26
27         elif delta < 0:
28             real_part = -self.b / (2 * self.a)

```

```
28         imag_part = math.sqrt(abs(delta)) / (2 * self.a)
29         x1 = complex(real_part, imag_part)
30         x2 = complex(real_part, -imag_part)
31         return x1, x2
32
33     else:
34         x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
35         x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
36         return x1, x2
37
38
39 class EquationSolver_Mutant_AOR:
40     def __init__(self, a, b, c):
41         self.a = a
42         self.b = b
43         self.c = c
44
45     def solve(self):
46         if not all(
47             isinstance(x, (int, float)) and not isinstance(x, bool)
48             for x in [self.a, self.b, self.c]
49         ):
50             raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
51 reale!")
52
53         if self.a == 0:
54             raise ValueError("Coeficientul a nu poate fi 0!")
55
56         # Mutant AOR: b**2 - 4*a*c devine b**2 + 4*a*c
57         delta = self.b ** 2 + 4 * self.a * self.c
58
59         if math.isclose(delta, 0.0, abs_tol=1e-9):
60             return -self.b / (2 * self.a)
61
62         elif delta < 0:
63             real_part = -self.b / (2 * self.a)
64             imag_part = math.sqrt(abs(delta)) / (2 * self.a)
65             x1 = complex(real_part, imag_part)
66             x2 = complex(real_part, -imag_part)
67             return x1, x2
68
69         else:
70             x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
71             x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
72             return x1, x2
73
74 class EquationSolver_Mutant_ROR:
75     def __init__(self, a, b, c):
76         self.a = a
77         self.b = b
78         self.c = c
79
80     def solve(self):
81         if not all(
82             isinstance(x, (int, float)) and not isinstance(x, bool)
83             for x in [self.a, self.b, self.c]
84         ):
85             raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
86 reale!")
87
88         if self.a == 0:
89             raise ValueError("Coeficientul a nu poate fi 0!")
90
91         # Mutant ROR: b**2 - 4*a*c devine b**2 - 4*a*c
92         delta = self.b ** 2 - 4 * self.a * self.c
93
94         if math.isclose(delta, 0.0, abs_tol=1e-9):
95             return -self.b / (2 * self.a)
96
97         elif delta < 0:
98             real_part = -self.b / (2 * self.a)
99             imag_part = math.sqrt(abs(delta)) / (2 * self.a)
100             x1 = complex(real_part, imag_part)
101             x2 = complex(real_part, -imag_part)
102             return x1, x2
103
104         else:
105             x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
106             x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
107             return x1, x2
```

```

85         raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
reale!")
86
87     if self.a == 0:
88         raise ValueError("Coeficientul a nu poate fi 0!")
89
90     delta = self.b ** 2 - 4 * self.a * self.c
91
92     if math.isclose(delta, 0.0, abs_tol=1e-9):
93         return -self.b / (2 * self.a)
94
95     # Mutant ROR: delta < 0 devine delta > 0
96     elif delta > 0:
97         real_part = -self.b / (2 * self.a)
98         imag_part = math.sqrt(abs(delta)) / (2 * self.a)
99         x1 = complex(real_part, imag_part)
100        x2 = complex(real_part, -imag_part)
101        return x1, x2
102
103    else:
104        x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
105        x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
106        return x1, x2
107
108
109 class EquationSolver_Mutant_LCR:
110     def __init__(self, a, b, c):
111         self.a = a
112         self.b = b
113         self.c = c
114
115     def solve(self):
116         # Mutant LCR: all(...) devine any(...)
117         if not any(
118             isinstance(x, (int, float)) and not isinstance(x, bool)
119             for x in [self.a, self.b, self.c]
120         ):
121             raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
reale!")
122
123         if self.a == 0:
124             raise ValueError("Coeficientul a nu poate fi 0!")
125
126         delta = self.b ** 2 - 4 * self.a * self.c
127
128         if math.isclose(delta, 0.0, abs_tol=1e-9):
129             return -self.b / (2 * self.a)
130
131         elif delta < 0:
132             real_part = -self.b / (2 * self.a)
133             imag_part = math.sqrt(abs(delta)) / (2 * self.a)
134             x1 = complex(real_part, imag_part)
135             x2 = complex(real_part, -imag_part)
136             return x1, x2
137
138     else:
139         x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
140         x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)

```



```

141         return x1, x2
142
143
144 class EquationSolver_Mutant_STD1:
145     def __init__(self, a, b, c):
146         self.a = a
147         self.b = b
148         self.c = c
149
150     def solve(self):
151         if not all(
152             isinstance(x, (int, float)) and not isinstance(x, bool)
153             for x in [self.a, self.b, self.c]
154         ):
155             raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
156 reale!")
157
158         if self.a == 0:
159             raise ValueError("Coeficientul a nu poate fi 0!")
160
161         delta = self.b ** 2 - 4 * self.a * self.c
162
163         if math.isclose(delta, 0.0, abs_tol=1e-9):
164             return -self.b / (2 * self.a)
165
166         elif delta < 0:
167             real_part = -self.b / (2 * self.a)
168             imag_part = math.sqrt(abs(delta)) / (2 * self.a)
169             x1 = complex(real_part, imag_part)
170             x2 = complex(real_part, -imag_part)
171
172             # Mutant STD-1: returnarea corecta este inlocuita cu None
173             return None
174
175         else:
176             x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
177             x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
178             return x1, x2
179
180 class EquationSolver_Mutant_STD2:
181     def __init__(self, a, b, c):
182         self.a = a
183         self.b = b
184         self.c = c
185
186     def solve(self):
187         if not all(
188             isinstance(x, (int, float)) and not isinstance(x, bool)
189             for x in [self.a, self.b, self.c]
190         ):
191             raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
192 reale!")
193
194         if self.a == 0:
195             raise ValueError("Coeficientul a nu poate fi 0!")
196
197         delta = self.b ** 2 - 4 * self.a * self.c

```

```

197
198     if math.isclose(delta, 0.0, abs_tol=1e-9):
199         # Mutant STD-2: -b / (2*a) devine -b / a
200         return -self.b / self.a
201
202     elif delta < 0:
203         real_part = -self.b / (2 * self.a)
204         imag_part = math.sqrt(abs(delta)) / (2 * self.a)
205         x1 = complex(real_part, imag_part)
206         x2 = complex(real_part, -imag_part)
207         return x1, x2
208
209     else:
210         x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
211         x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
212         return x1, x2

```

Secvență de cod 4.1: Implementarea clasei EquationSolver și a mutanților

Pentru a menține consistența cu implementarea originală, validarea coeficienților a fost păstrată identică în toate clasele, inclusiv în clasele mutante. Astfel, un coeficient este considerat valid doar dacă are tipul `int` sau `float`, dar nu are tipul `bool`. Această precizare este necesară deoarece, în Python, tipul `bool` este subclasă a tipului `int`, iar o verificare simplă de forma `isinstance(x, (int, float))` ar accepta valorile `True` și `False` ca fiind coeficienți numerici validați. Prin introducerea funcției auxiliare `este_coeficient_valid`, se evită duplicarea codului și se asigură că atât implementarea originală, cât și mutanții pornesc de la aceeași logică de validare.

4.4 Teste pentru implementarea originală și pentru mutanți

Pentru validarea implementării originale și pentru verificarea faptului că mutanții sunt detectați, au fost definite teste unitare folosind framework-ul `unittest`.

```

1 class TestEquationSolverMutationAdequacy(unittest.TestCase):
2
3     def test_invalid_types(self):
4         with self.assertRaises(ValueError):
5             EquationSolver("text", 2, 1).solve()
6
7         with self.assertRaises(ValueError):
8             EquationSolver(1, "text", 1).solve()
9
10        with self.assertRaises(ValueError):
11            EquationSolver(1, 2, "text").solve()
12
13        with self.assertRaises(ValueError):
14            EquationSolver(True, 2, 1).solve()
15
16    def test_a_zero(self):
17        with self.assertRaises(ValueError):
18            EquationSolver(0, 2, 1).solve()
19
20    def test_delta_negative(self):
21        result = EquationSolver(1, 1, 1).solve()
22
23        self.assertIsInstance(result, tuple)

```

```

24     self.assertEqual(len(result), 2)
25     self.assertIsInstance(result[0], complex)
26     self.assertIsInstance(result[1], complex)
27
28     self.assertAlmostEqual(result[0].real, -0.5)
29     self.assertAlmostEqual(result[1].real, -0.5)
30
31     def test_delta_zero(self):
32         result = EquationSolver(1, 2, 1).solve()
33
34         self.assertIsInstance(result, float)
35         self.assertAlmostEqual(result, -1.0)
36
37     def test_delta_positive(self):
38         result = EquationSolver(1, 5, 6).solve()
39
40         self.assertIsInstance(result, tuple)
41         self.assertEqual(len(result), 2)
42
43         self.assertAlmostEqual(result[0], -2.0)
44         self.assertAlmostEqual(result[1], -3.0)
45
46     def test_mutant_aor_is_killed(self):
47         original = EquationSolver(1, 5, 6).solve()
48         mutant = EquationSolver_Mutant_AOR(1, 5, 6).solve()
49
50         self.assertNotEqual(original, mutant)
51
52     def test_mutant_ror_is_killed(self):
53         original = EquationSolver(1, 2, 1).solve()
54         mutant = EquationSolver_Mutant_ROR(1, 2, 1).solve()
55
56         self.assertNotEqual(original, mutant)
57
58     def test_mutant_lcr_is_killed(self):
59         with self.assertRaises(Exception):
60             EquationSolver_Mutant_LCR("text", 2, 1).solve()
61
62     def test_mutant_std1_is_killed(self):
63         original = EquationSolver(1, 1, 1).solve()
64         mutant = EquationSolver_Mutant_STD1(1, 1, 1).solve()
65
66         self.assertNotEqual(original, mutant)
67
68     def test_mutant_std2_is_killed(self):
69         original = EquationSolver(1, 2, 1).solve()
70         mutant = EquationSolver_Mutant_STD2(1, 2, 1).solve()
71
72         self.assertNotEqual(original, mutant)
73
74
75 if __name__ == "__main__":
76     unittest.main(verbosity=2)

```

Secvență de cod 4.2: Teste unitare pentru EquationSolver și pentru mutanți

4.4.1 Strong vs Weak Mutation

Un test omoară un mutant dacă diferența dintre programul original și mutant este observabilă.

- **Weak mutation:** diferența apare doar în starea internă a programului.
- **Strong mutation:** diferența se propagă până la ieșire și este observabilă în rezultatul final.

În cadrul acestei lucrări se utilizează strong mutation, deoarece verificarea rezultatelor finale garantează detectarea efectivă a erorilor.

4.5 Analiza mutanților

Pentru a evalua eficiența setului de teste, fiecare mutant este executat separat împreună cu testele definite pentru clasa `EquationSolver`. Rezultatele obținute sunt sintetizate în Tabela 4.2.

Tabela 4.2: Rezultatul execuției testelor asupra mutanților

Mutant	Status	Motiv
AOR	Ucis	Testele pentru ecuații cu $\Delta > 0$ sau $\Delta < 0$ detectează imediat calculul eronat al discriminantului.
ROR	Ucis	Este detectat prin testul explicit pentru cazul de frontieră $\Delta = 0$.
LCR	Ucis	Testele pentru tipuri invalide detectează lipsa validării corecte a coeficienților.
STD-1	Ucis	Testele pentru cazul $\Delta < 0$ detectează faptul că metoda nu mai returnează rădăcinile complexe.
STD-2	Ucis	Este detectat prin testul care verifică exact valoarea rădăcinii duble pentru cazul $\Delta = 0$.

În cazul mutantului de tip ROR, comportamentul programului este modificat doar pentru valoarea de frontieră $\Delta = 0$. Dacă setul de teste conține un caz care verifică explicit această situație, mutantul este detectat și considerat ucis. În absența unui astfel de test, mutantul poate supraviețui, deoarece pentru valori negative ale discriminantului condiția $\Delta \leq 0$ rămâne adevărată și programul se comportă similar cu versiunea originală.

4.5.1 Condiții pentru detectarea mutanților

Pentru ca un test să detecteze un mutant, trebuie îndeplinite trei condiții:

- **Reachability:** instrucțiunea modificată trebuie să fie executată.
- **State infection:** modificarea trebuie să afecteze starea internă a programului.
- **State propagation:** diferența trebuie să se propage până la ieșire.

Dacă una dintre aceste condiții nu este îndeplinită, mutantul poate supraviețui, chiar dacă există o eroare în cod.

4.6 Rezultatele rulării testelor

Pentru a valida comportamentul implementării originale și pentru a verifica dacă mutanții sunt detectați, testele au fost rulate folosind framework-ul `unittest` din Python. În cazul implementării corecte, toate testele definite pentru clasa `EquationSolver` trec cu succes.

Un exemplu de rezultat obținut la rularea testelor este prezentat mai jos:

```
test_invalid_types ... ok
test_a_zero ... ok
test_delta_negative ... ok
test_delta_zero ... ok
test_delta_positive ... ok
test_mutant_aor_is_killed ... ok
test_mutant_ror_is_killed ... ok
test_mutant_std1_is_killed ... ok
test_mutant_std2_is_killed ... ok
test_mutant_lcr_is_killed ... ok
```

```
-----
Ran 10 tests in 0.001s
```

OK

Rezultatele obținute confirmă faptul că testele definite sunt suficiente pentru a detecta modificările introduse prin operatorii de mutație analizați.

4.6.1 Mutanți echivalenți

Un mutant este considerat echivalent dacă produce aceleași rezultate ca programul original pentru orice date de intrare.

Acești mutanți nu pot fi detectați prin teste, deoarece nu există diferențe observabile între comportamentul lor și cel al programului original.

Determinarea automată a mutanților echivalenți este, în general, o problemă nedecidabilă, motiv pentru care identificarea acestora se realizează, în practică, prin analiză manuală.

4.7 Concluzii

Analiza pe mutanți evidențiază faptul că un set de teste bun trebuie să acopere nu doar toate ramurile de execuție, ci și valorile de frontieră și rezultatele numerice exacte returnate de program.

Pentru a evalua eficiența setului de teste se poate calcula scorul de mutație (Mutation Score), definit ca raportul dintre numărul de mutanți uciși și numărul total de mutanți generați:

$$MS = \frac{M_{killed}}{M_{total}}$$

unde M_{killed} reprezintă numărul mutanților detectați de teste, iar M_{total} reprezintă numărul total de mutanți generați.

O formulare echivalentă utilizată în literatura de specialitate este:

$$MS = \frac{D}{L + D}$$

unde:

- D reprezintă numărul mutanților uciși;
- L reprezintă numărul mutanților nedetectați (live mutants).

În cazul analizat:

$$MS = \frac{5}{5} = 100\%$$

Acest rezultat arată că setul de teste este capabil să detecteze toți mutanții considerați în experiment.

În concluzie, analiza realizată demonstrează că setul de teste implementat pentru clasa `EquationSolver` este capabil să detecteze majoritatea modificărilor introduse în cod prin operatorii de mutație. Prin combinarea testării funcționale, a testării structurale și a testării pe mutanți, se obține o validare mai riguroasă a corectitudinii și robusteții algoritmului.

De asemenea, rezultatele obținute sunt în concordanță cu ipoteza competent programmer hypothesis, conform căreia erorile introduse în program sunt, de regulă, mici variații față de implementarea corectă și pot fi detectate prin mutații simple.

În plus, efectul de cuplare (coupling effect) sugerează că testele capabile să detecteze mutanți de ordin 1 sunt, în general, suficiente pentru a detecta și erori mai complexe.

Rezultatele obținute sunt în concordanță cu ipoteza competent programmer hypothesis, conform căreia erorile sunt, în general, mici deviații de la implementarea corectă.

De asemenea, efectul de cuplare (coupling effect) sugerează că testele care detectează mutanți simpli sunt suficiente pentru a detecta și erori mai complexe.

4.8 Testare pe mutanți folosind Cosmic Ray

Pentru completarea analizei de mutation testing, am utilizat și instrumentul `Cosmic Ray`, un tool de testare pe mutanți pentru programe Python. Acesta generează automat variante modificate ale codului sursă, numite mutanți, aplicând operatori de mutație asupra expresiilor, condițiilor și operatorilor existenți în program.

Scopul utilizării `Cosmic Ray` este evaluarea calității setului de teste. Dacă testele reușesc să distingă programul original de o versiune modificată, mutantul este considerat *killed*. Dacă testele continuă să treacă și pe varianta modificată, mutantul este considerat *survived*, ceea ce poate indica fie o lipsă în setul de teste, fie existența unui mutant echivalent.

Pentru clasa `EquationSolver`, `Cosmic Ray` poate genera mutanți asupra expresiilor matematice și condițiilor principale din metoda `solve()`, precum:

- modificarea expresiei discriminantului `b ** 2 - 4 * a * c`;
- modificarea condiției `a == 0`;
- modificarea condiției `delta < 0`;

- modificarea verificării `math.isclose(delta, 0.0);`
- modificarea operatorilor aritmetici din formulele rădăcinilor.

Astfel, Cosmic Ray verifică dacă testele existente sunt suficient de puternice pentru a detecta modificări subtile în logica programului. Spre deosebire de acoperirea codului, care arată doar dacă liniile au fost executate, mutation testing verifică și dacă testele validează corect rezultatele obținute.

4.8.1 Rularea testării pe mutanți cu Cosmic Ray

Pentru completarea analizei de mutation testing, am utilizat instrumentul Cosmic Ray, un tool automatizat pentru testarea pe mutanți în programe Python. Acesta generează versiuni modificate ale codului sursă, numite mutanți, prin aplicarea unor operatori de mutație asupra expresiilor, operatorilor aritmetici, operatorilor de comparație și valorilor numerice.

În cadrul proiectului, fișierul supus mutației a fost `equation_solver.py`, iar pentru fiecare mutant generat a fost rulată suita de teste definită în `test_equation_solver.py`. Configurarea utilizată a fost realizată prin fișierul `config.toml`, iar rularea s-a efectuat prin comenzile:

```
cosmic-ray init config.toml session.sqlite
cosmic-ray exec config.toml session.sqlite
cr-report session.sqlite --show-pending
```

Scopul acestei etape a fost verificarea capacității testelor de a detecta defecte artificiale introduse în cod. Dacă testele eșuează pentru un mutant, acesta este considerat *killed*, deoarece modificarea a fost detectată. Dacă testele trec în continuare, mutantul este considerat *survived*, ceea ce poate indica fie o zonă insuficient testată, fie existența unui mutant echivalent.

Rezultatul final obținut în urma rulării instrumentului Cosmic Ray a fost:

```
(.venv) C:\Users\Adelina\PycharmProjects\PythonProject> cr-report session.sqlite
```

```
total jobs: 234
complete: 234 (100.00%)
surviving mutants: 9 (3.85%)
```

Raportul Cosmic Ray indică un total de 234 de mutanți generați, dintre care 9 au supraviețuit. Prin urmare, numărul de mutanți omorâți de suita de teste este:

$$M_{killed} = 234 - 9 = 225$$

Dacă nu eliminăm din calcul mutanții echivalenți, scorul brut de mutație este:

$$MutationScore_{brut} = \frac{M_{killed}}{M_{total}} \cdot 100 = \frac{225}{234} \cdot 100 \approx 96.15\%$$

Acest scor este unul ridicat și arată că suita de teste detectează majoritatea modificărilor artificiale introduse în cod. Totuși, deoarece unii mutanți supraviețuitori pot fi echivalenți cu programul original pentru domeniul de intrări testat, scorul brut trebuie interpretat cu atenție.

În cazul în care identificăm manual un număr M_{equiv} de mutanți echivalenți, formula ajustată a scorului de mutație devine:

$$MutationScore_{ajustat} = \frac{M_{killed}}{M_{total} - M_{equiv}} \cdot 100$$

În cadrul acestui proiect, păstrăm valoarea de 96.15% ca scor brut, deoarece mutanții echivalenți nu au fost eliminați automat din raportul Cosmic Ray. Aceștia sunt discutați separat în analiza mutanților supraviețuitori.

Operatorii de mutație aplicați de Cosmic Ray au inclus, printre altele:

- înlocuirea operatorilor aritmetici, de exemplu +, -, *, /;
- înlocuirea operatorilor de comparație, de exemplu ==, <, <=;
- modificarea operatorilor unari;
- înlocuirea unor constante numerice prin operatorul NumberReplacer.

În cazul clasei `EquationSolver`, acești operatori sunt relevanți deoarece algoritmul depinde direct de calcule matematice precum discriminantul $\Delta = b^2 - 4ac$, verificarea condiției $\Delta < 0$, verificarea cazului $\Delta = 0$ și formulele de calcul ale rădăcinilor.

Mutanții supraviețuitori au fost generați în special prin înlocuirea operatorului de înmulțire cu operatori precum împărțire, împărțire întreagă sau ridicare la putere. Aceștia indică faptul că anumite expresii matematice pot produce rezultate suficient de apropiate pentru unele cazuri de test sau pot reprezenta mutanți echivalenți în raport cu datele de test folosite. Din acest motiv, mutanții supraviețuitori trebuie analizați separat, pentru a decide dacă este necesară adăugarea unor teste suplimentare.

Comparativ cu acoperirea codului, care a indicat o acoperire de 100%, testarea pe mutanți oferă o perspectivă mai riguroasă. Coverage-ul confirmă că instrucțiunile au fost executate, în timp ce Cosmic Ray verifică dacă testele sunt suficient de puternice pentru a detecta modificări greșite ale logicii programului. Prin urmare, rezultatul obținut cu Cosmic Ray confirmă calitatea ridicată a suitei de teste, dar evidențiază și existența unor mutații care pot fi investigate suplimentar.

4.8.2 Analiza mutanților supraviețuitori

În raportul generat de Cosmic Ray au fost identificați 9 mutanți supraviețuitori. Aceștia provin din operatori de tip `ReplaceBinaryOperator`, mai exact din mutații care înlocuiesc operatorul de înmulțire cu operatori precum împărțirea, împărțirea întreagă sau ridicarea la putere.

Această valoare este consistentă cu raportul final Cosmic Ray, unde cei 9 mutanți supraviețuitori reprezintă 3.85% din totalul de 234 de mutanți generați. Acești mutanți nu trebuie interpretați automat ca defecte rămase în implementare. Ei pot indica fie cazuri insuficient acoperite de datele de test, fie mutanți echivalenți, adică variante modificate ale programului care produc același comportament observabil pentru setul de intrări analizat. Exemple de astfel de mutații sunt:

- `ReplaceBinaryOperator_Mul_Div`;
- `ReplaceBinaryOperator_Mul_FloorDiv`;
- `ReplaceBinaryOperator_Mul_Pow`.

Faptul că acești mutanți au supraviețuit nu înseamnă automat că implementarea este greșită, ci arată că testele existente nu au reușit să distingă în toate cazurile programul original de varianta modificată. O posibilă explicație este faptul că unele mutații apar în expresii pentru care valorile alese în testele curente produc rezultate apropiate sau nu modifică suficient comportamentul observabil.

Pentru îmbunătățirea suitei de teste, se pot adăuga cazuri suplimentare cu valori mai variate pentru coeficienții a , b și c , în special valori care modifică semnificativ numitorul $2a$, discriminantul și radicalul utilizat în calculul rădăcinilor.

Capitolul 5

Raport privind utilizarea IA pentru generarea testelor

5.1 Introducere

În cadrul acestei etape a proiectului, am utilizat un instrument de inteligență artificială pentru a genera o clasă completă de teste automate pentru o clasă Python numită `EquationSolver`. Scopul utilizării AI a fost obținerea rapidă a unui set coerent de teste unitare, scrise cu framework-ul `unittest`, care să verifice funcționalitatea metodei `solve()` pentru ecuații de gradul al II-lea.

Clasa analizată primește trei coeficienți, `a`, `b` și `c`, corespunzători ecuației:

$$ax^2 + bx + c = 0$$

Metoda `solve()` validează coeficienții, calculează discriminantul și returnează rădăcinile ecuației în funcție de valoarea acestuia. Pentru realizarea testării, am formulat un prompt detaliat, în care am specificat atât comportamentul așteptat al clasei, cât și tipurile de acoperire care trebuiau verificate.

5.2 Promptul utilizat

```
1 \section{Promptul folosit}
2
3 \begin{lstlisting}[
4 frame=single,
5 breaklines=true,
6 basicstyle=\ttfamily\small,
7 ]
8 Ai urmatoarea clasa Python EquationSolver:
9
10 import math
11
12
13 class EquationSolver:
14     def __init__(self, a, b, c):
15         self.a = a
16         self.b = b
```

```
17     self.c = c
18
19     def solve(self):
20         if not all(isinstance(x, (int, float)) and not isinstance(x,
21 bool) for x in [self.a, self.b, self.c]):
22             raise ValueError("Coeficientii ecuatiei trebuie sa fie
23 numere reale!")
24
25         if self.a == 0:
26             raise ValueError("Coeficientul a nu poate fi 0!")
27
28         delta = self.b ** 2 - 4 * self.a * self.c
29
30         if delta < 0:
31             real_part = -self.b / (2 * self.a)
32             imag_part = math.sqrt(abs(delta)) / (2 * self.a)
33             x1 = complex(real_part, imag_part)
34             x2 = complex(real_part, -imag_part)
35             return x1, x2
36
37         elif math.isclose(delta, 0.0, abs_tol=1e-9):
38             return -self.b / (2 * self.a)
39
40         else:
41             x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
42             x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
43             return x1, x2
```

44 Genereaza o clasa completa de teste Python folosind unittest,
45 numita TestEquationSolver, cu teste separate pentru:

- 46 1. statement coverage;
- 47 2. branch coverage;
- 48 3. condition coverage;
- 49 4. boundary value analysis;
- 50 5. equivalence partitioning;
- 51 6. category partitioning;
- 52 7. path coverage;
- 53 8. mutation testing / killMutants.

54 Include cazuri pentru:

- 55 - coeficient invalid a, b sau c;
- 56 - valori de tip bool, de exemplu True sau False;
- 57 - cazul a == 0;
- 58 - delta < 0;
- 59 - delta == 0;
- 60 - delta > 0;
- 61 - valori de frontiera pentru a aproape de 0: a = -0.001 si a = 0.001;
- 62 - valori de frontiera pentru delta aproape de 0: b = 1.999 si b =
- 63 2.001, cu a = 1 si c = 1.

```
65 Pentru partea de console output, foloseste io.StringIO si
66 contextlib.redirect_stdout pentru a verifica faptul ca metoda solve()
67 nu afiseaza nimic la consola.
68
69 Cerinte suplimentare:
70 - Codul trebuie sa fie complet si rulabil direct in PyCharm.
71 - Nu folosi pytest, foloseste doar unittest.
72 - Nu modifica implementarea clasei originale.
73 - Include if __name__ == "__main__": unittest.main(verbosity=2)
74 - Scrie comentarii scurte in cod pentru fiecare grup de teste.
```

5.3 Output-ul generat de AI

Ca răspuns la prompt, instrumentul AI a generat o clasă completă de teste Python, denumită TestEquationSolver, folosind biblioteca standard unittest. Codul generat este rulabil direct în PyCharm și include instrucțiunea:

```
1 if __name__ == "__main__":
2     unittest.main(verbosity=2)
```

Output-ul a început prin importarea bibliotecilor necesare:

5.4 Clasa de teste generată

```
1 import math
2 import unittest
3 import io
4 from contextlib import redirect_stdout
5
6
7 class EquationSolver:
8     def __init__(self, a, b, c):
9         self.a = a
10        self.b = b
11        self.c = c
12
13    def solve(self):
14        if not all(isinstance(x, (int, float)) and not isinstance(x,
15        bool) for x in [self.a, self.b, self.c]):
16            raise ValueError("Coeficientii ecuatiei trebuie sa fie
17        numere reale!")
18
19        if self.a == 0:
20            raise ValueError("Coeficientul a nu poate fi 0!")
21
22        delta = self.b ** 2 - 4 * self.a * self.c
23
24        if delta < 0:
25            real_part = -self.b / (2 * self.a)
26            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
```

```
25         x1 = complex(real_part, imag_part)
26         x2 = complex(real_part, -imag_part)
27         return x1, x2
28
29     elif math.isclose(delta, 0.0, abs_tol=1e-9):
30         return -self.b / (2 * self.a)
31
32     else:
33         x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
34         x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
35         return x1, x2
36
37
38 class TestEquationSolver(unittest.TestCase):
39
40     # Helper pentru compararea numerelor complexe
41     def assertComplexAlmostEqual(self, actual, expected, places=7):
42         self.assertAlmostEqual(actual.real, expected.real, places=
43 places)
44         self.assertAlmostEqual(actual.imag, expected.imag, places=
45 places)
46
47     # 1. Statement coverage
48     def test_statement_coverage_delta_positive(self):
49         solver = EquationSolver(1, -5, 6)
50         result = solver.solve()
51
52         self.assertIsInstance(result, tuple)
53         self.assertEqual(len(result), 2)
54         self.assertAlmostEqual(result[0], 3.0)
55         self.assertAlmostEqual(result[1], 2.0)
56
57     def test_statement_coverage_delta_negative(self):
58         solver = EquationSolver(1, 2, 5)
59         x1, x2 = solver.solve()
60
61         self.assertComplexAlmostEqual(x1, complex(-1.0, 2.0))
62         self.assertComplexAlmostEqual(x2, complex(-1.0, -2.0))
63
64     def test_statement_coverage_delta_zero(self):
65         solver = EquationSolver(1, 2, 1)
66         result = solver.solve()
67
68         self.assertAlmostEqual(result, -1.0)
69
70     # 2. Branch coverage
71     def test_branch_coverage_invalid_coefficient(self):
72         with self.assertRaises(ValueError) as context:
73             EquationSolver("1", 2, 1).solve()
74
75         self.assertEqual(
```

```
74         str(context.exception),
75         "Coeficientii ecuatiei trebuie sa fie numere reale!"
76     )
77
78     def test_branch_coverage_a_zero(self):
79         with self.assertRaises(ValueError) as context:
80             EquationSolver(0, 2, 1).solve()
81
82         self.assertEqual(
83             str(context.exception),
84             "Coeficientul a nu poate fi 0!"
85         )
86
87     def test_branch_coverage_all_delta_cases(self):
88         negative_delta = EquationSolver(1, 0, 1).solve()
89         zero_delta = EquationSolver(1, 2, 1).solve()
90         positive_delta = EquationSolver(1, -3, 2).solve()
91
92         self.assertIsInstance(negative_delta[0], complex)
93         self.assertAlmostEqual(zero_delta, -1.0)
94         self.assertAlmostEqual(positive_delta[0], 2.0)
95         self.assertAlmostEqual(positive_delta[1], 1.0)
96
97     # 3. Condition coverage
98     def test_condition_coverage_invalid_a(self):
99         with self.assertRaises(ValueError):
100             EquationSolver("invalid", 2, 1).solve()
101
102     def test_condition_coverage_invalid_b(self):
103         with self.assertRaises(ValueError):
104             EquationSolver(1, "invalid", 1).solve()
105
106     def test_condition_coverage_invalid_c(self):
107         with self.assertRaises(ValueError):
108             EquationSolver(1, 2, "invalid").solve()
109
110     def test_condition_coverage_bool_a(self):
111         with self.assertRaises(ValueError):
112             EquationSolver(True, 2, 1).solve()
113
114     def test_condition_coverage_bool_b(self):
115         with self.assertRaises(ValueError):
116             EquationSolver(1, False, 1).solve()
117
118     def test_condition_coverage_bool_c(self):
119         with self.assertRaises(ValueError):
120             EquationSolver(1, 2, True).solve()
121
122     def test_condition_coverage_valid_int_and_float_values(self):
123         result = EquationSolver(1.0, -3, 2.0).solve()
124
```

```
125         self.assertAlmostEqual(result[0], 2.0)
126         self.assertAlmostEqual(result[1], 1.0)
127
128     # 4. Boundary value analysis
129     def test_boundary_value_a_negative_near_zero(self):
130         solver = EquationSolver(-0.001, 0, 1)
131         x1, x2 = solver.solve()
132
133         self.assertAlmostEqual(x1, -31.6227766017, places=6)
134         self.assertAlmostEqual(x2, 31.6227766017, places=6)
135
136     def test_boundary_value_a_positive_near_zero(self):
137         solver = EquationSolver(0.001, 0, 1)
138         x1, x2 = solver.solve()
139
140         self.assertComplexAlmostEqual(x1, complex(0.0, 31.6227766017),
141 , places=6)
142         self.assertComplexAlmostEqual(x2, complex(0.0,
143 -31.6227766017), places=6)
144
145     def test_boundary_value_delta_near_zero_negative(self):
146         solver = EquationSolver(1, 1.999, 1)
147         x1, x2 = solver.solve()
148
149         self.assertIsInstance(x1, complex)
150         self.assertIsInstance(x2, complex)
151         self.assertAlmostEqual(x1.real, -0.9995, places=6)
152         self.assertAlmostEqual(x2.real, -0.9995, places=6)
153
154     def test_boundary_value_delta_near_zero_positive(self):
155         solver = EquationSolver(1, 2.001, 1)
156         x1, x2 = solver.solve()
157
158         self.assertIsInstance(x1, float)
159         self.assertIsInstance(x2, float)
160         self.assertAlmostEqual(x1, -0.9688732708, places=6)
161         self.assertAlmostEqual(x2, -1.0321267292, places=6)
162
163     # 5. Equivalence partitioning
164     def test_equivalence_partition_valid_real_roots(self):
165         x1, x2 = EquationSolver(1, -4, 3).solve()
166
167         self.assertAlmostEqual(x1, 3.0)
168         self.assertAlmostEqual(x2, 1.0)
169
170     def test_equivalence_partition_valid_double_root(self):
171         result = EquationSolver(4, 4, 1).solve()
172
173         self.assertAlmostEqual(result, -0.5)
174
175     def test_equivalence_partition_valid_complex_roots(self):
```

```

174     x1, x2 = EquationSolver(1, 0, 4).solve()
175
176     self.assertEqual(x1, complex(0.0, 2.0))
177     self.assertEqual(x2, complex(0.0, -2.0))
178
179     def test_equivalence_partition_invalid_non_numeric(self):
180         invalid_values = [
181             ("a", 1, 1),
182             (1, "b", 1),
183             (1, 1, "c"),
184             (None, 1, 1),
185             (1, [], 1),
186             (1, 1, {}))
187         ]
188
189         for a, b, c in invalid_values:
190             with self.subTest(a=a, b=b, c=c):
191                 with self.assertRaises(ValueError):
192                     EquationSolver(a, b, c).solve()
193
194     # 6. Category partitioning
195     def test_category_partitioning_coefficient_types(self):
196         categories = [
197             (1, 2, 1, True),
198             (1.0, 2.0, 1.0, True),
199             (True, 2, 1, False),
200             (1, False, 1, False),
201             (1, 2, True, False),
202             ("1", 2, 1, False)
203         ]
204
205         for a, b, c, should_be_valid in categories:
206             with self.subTest(a=a, b=b, c=c):
207                 if should_be_valid:
208                     self.assertIsNotNone(EquationSolver(a, b, c).
209 solve())
210                 else:
211                     with self.assertRaises(ValueError):
212                         EquationSolver(a, b, c).solve()
213
214     def test_category_partitioning_delta_categories(self):
215         categories = [
216             (1, 0, 1, "negative_delta"),
217             (1, 2, 1, "zero_delta"),
218             (1, -3, 2, "positive_delta")
219         ]
220
221         for a, b, c, category in categories:
222             with self.subTest(category=category):
223                 result = EquationSolver(a, b, c).solve()

```



```

224         if category == "negative_delta":
225             self.assertIsInstance(result[0], complex)
226             self.assertIsInstance(result[1], complex)
227         elif category == "zero_delta":
228             self.assertIsInstance(result, float)
229             self.assertAlmostEqual(result, -1.0)
230         else:
231             self.assertIsInstance(result, tuple)
232             self.assertAlmostEqual(result[0], 2.0)
233             self.assertAlmostEqual(result[1], 1.0)
234
235     # 7. Path coverage
236     def test_path_coverage_invalid_type_path(self):
237         with self.assertRaises(ValueError):
238             EquationSolver([], 2, 1).solve()
239
240     def test_path_coverage_a_zero_path(self):
241         with self.assertRaises(ValueError):
242             EquationSolver(0, 2, 1).solve()
243
244     def test_path_coverage_delta_negative_path(self):
245         result = EquationSolver(1, 2, 10).solve()
246
247         self.assertIsInstance(result, tuple)
248         self.assertIsInstance(result[0], complex)
249         self.assertIsInstance(result[1], complex)
250
251     def test_path_coverage_delta_zero_path(self):
252         result = EquationSolver(9, 6, 1).solve()
253
254         self.assertAlmostEqual(result, -1 / 3)
255
256     def test_path_coverage_delta_positive_path(self):
257         x1, x2 = EquationSolver(2, -7, 3).solve()
258
259         self.assertAlmostEqual(x1, 3.0)
260         self.assertAlmostEqual(x2, 0.5)
261
262     # 8. Mutation testing / killMutants
263     def test_kill_mutant_accepts_bool_as_number(self):
264         with self.assertRaises(ValueError):
265             EquationSolver(True, 2, 1).solve()
266
267     def test_kill_mutant_allows_a_zero(self):
268         with self.assertRaises(ValueError):
269             EquationSolver(0, 1, 1).solve()
270
271     def test_kill_mutant_delta_sign_logic(self):
272         complex_result = EquationSolver(1, 0, 1).solve()
273         real_result = EquationSolver(1, -3, 2).solve()
274

```

```
275     self.assertIsInstance(complex_result[0], complex)
276     self.assertAlmostEqual(real_result[0], 2.0)
277     self.assertAlmostEqual(real_result[1], 1.0)
278
279     def test_kill_mutant_double_root_returns_single_value(self):
280         result = EquationSolver(1, -2, 1).solve()
281
282         self.assertNotIsInstance(result, tuple)
283         self.assertAlmostEqual(result, 1.0)
284
285     def test_kill_mutant_wrong_quadratic_formula(self):
286         x1, x2 = EquationSolver(1, -5, 6).solve()
287
288         self.assertAlmostEqual(x1, 3.0)
289         self.assertAlmostEqual(x2, 2.0)
290         self.assertNotAlmostEqual(x1, -3.0)
291         self.assertNotAlmostEqual(x2, -2.0)
292
293     # Verificare console output
294     def test_solve_does_not_print_anything_for_valid_case(self):
295         output = io.StringIO()
296
297         with redirect_stdout(output):
298             result = EquationSolver(1, -5, 6).solve()
299
300         self.assertEqual(output.getvalue(), "")
301         self.assertAlmostEqual(result[0], 3.0)
302         self.assertAlmostEqual(result[1], 2.0)
303
304     def test_solve_does_not_print_anything_for_invalid_case(self):
305         output = io.StringIO()
306
307         with redirect_stdout(output):
308             with self.assertRaises(ValueError):
309                 EquationSolver("invalid", 2, 1).solve()
310
311         self.assertEqual(output.getvalue(), "")
312
313
314 if __name__ == "__main__":
315     unittest.main(verbosity=2)
```

5.5 Tehnicile de testare acoperite

5.5.1 Statement coverage

Pentru statement coverage, testele generate urmăresc executarea principalelor instrucțiuni din metoda `solve()`. Sunt incluse cazuri pentru coeficienți invalizi, $a == 0$, discriminant negativ, discriminant egal cu zero și discriminant pozitiv.

5.5.2 Branch coverage

Pentru branch coverage, output-ul conține teste care verifică fiecare ramură logică importantă a metodei. Astfel, sunt acoperite atât ramura în care validarea tipurilor eșuează, cât și ramura în care validarea reușește. De asemenea, sunt testate separat ramurile pentru $\Delta < 0$, $\Delta \approx 0$ și $\Delta > 0$.

5.5.3 Condition coverage

Pentru condition coverage, AI a generat teste separate pentru invalidarea fiecărui coeficient: `a`, `b` și `c`. De asemenea, au fost incluse teste speciale pentru valorile de tip `bool`, cum ar fi `True` și `False`, deoarece acestea nu trebuie acceptate de implementare.

5.5.4 Boundary value analysis

Pentru analiza valorilor de frontieră, au fost generate teste pentru valori ale lui `a` foarte apropiate de zero:

- `a = -0.001`;
- `a = 0.001`;
- `a = 0`.

De asemenea, au fost testate valori ale lui `b` care fac discriminantul foarte apropiat de zero:

- `b = 1.999`, cu `a = 1` și `c = 1`;
- `b = 2.001`, cu `a = 1` și `c = 1`.

Aceste teste sunt importante deoarece verifică stabilitatea metodei în jurul limitelor unde comportamentul se schimbă.

5.5.5 Equivalence partitioning

Pentru equivalence partitioning, AI a împărțit datele de intrare în clase de echivalență. Au fost testate coeficiente valide care duc la discriminant pozitiv, zero sau negativ, dar și clase invalide, precum valori de tip `str`, `None` sau `list`.

5.5.6 Category partitioning

Pentru category partitioning, testele au fost grupate pe categorii de intrări: coeficienți numerici valizi, coeficienți invalizi, valori booleene și cazul special în care `a` este zero.

5.5.7 Path coverage

Pentru path coverage, output-ul generat verifică traseele principale posibile prin metoda `solve()`: traseul pentru tip invalid, traseul pentru `a == 0`, traseul pentru discriminant negativ, traseul pentru discriminant zero și traseul pentru discriminant pozitiv.

5.6 Testarea output-ului în consolă

O cerință suplimentară a fost verificarea faptului că metoda `solve()` nu afișează nimic la consolă. Pentru aceasta, output-ul AI a folosit `io.StringIO` împreună cu `redirect_stdout`:

```

1 def test_solve_does_not_print_anything_to_console(self):
2     output = io.StringIO()
3
4     with redirect_stdout(output):
5         result = EquationSolver(1, -3, 2).solve()
6
7     self.assertEqual(output.getvalue(), "")
8     self.assertIsInstance(result, tuple)
9     self.assertEqual(len(result), 2)

```

Acest test confirmă faptul că metoda returnează rezultatul prin valoare, fără a folosi funcția `print()`.

5.7 Mutation testing

O parte importantă a output-ului generat este reprezentată de testele de tip mutation testing. În prompt au fost specificate cinci clase mutante:

- EquationSolver_Mutant_AOR;
- EquationSolver_Mutant_ROR;
- EquationSolver_Mutant_LCR;
- EquationSolver_Mutant_STD1;
- EquationSolver_Mutant_STD2.

Pentru fiecare mutant, AI a generat câte un test de tip `killMutants`, care verifică faptul că mutantul este detectat de cel puțin un caz de test. De exemplu, pentru mutantul AOR, care modifică discriminantul din scădere în adunare, testul folosește un caz în care implementarea originală ar trebui să returneze rădăcini complexe:

```

1 def test_kill_mutant_AOR_discriminant_plus_instead_of_minus(self):
2     with self.assertRaises(AssertionError):
3         self.assert_complex_roots(
4             EquationSolver_Mutant_AOR,
5             1,
6             0,
7             1,
8             expected_real=0.0,
9             expected_imag=1.0,
10        )

```

Acest test demonstrează că rezultatul mutantului diferă de rezultatul așteptat al implementării corecte, deci mutantul este detectat.

5.8 Observații asupra rezultatului generat

Output-ul generat de AI respectă cerințele principale din prompt. Testele sunt scrise folosind `unittest`, folosesc metode de aserțiune clare și sunt organizate pe categorii de

testare. Sunt utilizate `assertRaises` pentru excepții, `assertIsInstance` pentru verificarea tipurilor, `assertAlmostEqual` pentru valori reale sau complexe și `assertEqual` pentru verificări exacte.

Un avantaj al output-ului este faptul că include metode helper, ceea ce face codul mai ușor de citit și de întreținut. În loc ca aceleași verificări să fie repetate în fiecare test, acestea sunt centralizate în metode precum `assert_real_roots`, `assert_single_real_root` și `assert_complex_roots`.

Totuși, un aspect care trebuie verificat înainte de rularea efectivă este modul în care sunt importate clasele mutante. În funcție de structura proiectului, acestea pot fi definite fie în același fișier cu `EquationSolver`, fie într-un fișier separat, de exemplu `equation_solver_mutants.py`. Importurile trebuie adaptate în funcție de organizarea reală a fișierelor din proiect.

5.8.1 Comparatie între testele realizate manual și testele generate de AI

Pentru a evalua utilitatea instrumentului AI în procesul de testare, am comparat testele generate automat cu testele realizate manual în cadrul proiectului. Testele manuale au fost construite pornind de la tehnicile studiate la curs și seminar, precum partiționarea în clase de echivalență, analiza valorilor de frontieră, partiționarea în categorii, analiza fluxului de control și testarea pe mutanți.

Pe de altă parte, testele generate de AI au avut rolul de a propune rapid cazuri de test pentru funcționalitatea metodei `solve()`, acoperind scenarii importante precum coeficienți invalizi, cazul $a = 0$, discriminant negativ, discriminant nul și discriminant pozitiv.

Criteriu	Testele realizate manual	Testele generate de AI
Metodă de proiectare	Au fost construite sistematic, pe baza tehnicilor de testare studiate.	Au fost generate automat, pe baza promptului oferit.
Clase de echivalență	Acoperă explicit clasele valide și invalide: tipuri greșite, $a = 0$, $\Delta < 0$, $\Delta = 0$, $\Delta > 0$.	Acoperă cazurile principale, dar fără o justificare formală completă a claselor.
Valori de frontieră	Sunt analizate explicit frontierele $a = 0$ și $\Delta = 0$, inclusiv valori apropiate de acestea.	Poate propune teste de frontieră, dar nu întotdeauna le organizează conform metodei AVF.
Partiționare în categorii	Include categorii, alternative, constrângeri și cazuri de test derivate logic.	Nu oferă automat o structură completă de category partitioning decât dacă este cerută explicit.
Testare structurală	Include CFG, coverage, condiții, drumuri independente și complexitate ciclomatică.	Poate genera teste utile, dar nu construiește automat analiza structurală completă.
Mutation testing	Include atât mutanți definiți manual, cât și rulare automată cu Cosmic Ray.	Poate propune idei de mutanți, dar rezultatele trebuie validate prin rulări reale.
Rigoare teoretică	Ridică, deoarece fiecare test este justificat printr-o tehnică de testare.	Medie, deoarece output-ul depinde mult de claritatea promptului.
Viteză de generare	Necesită mai mult timp pentru analiză și justificare.	Generează rapid un set inițial de teste.
Fiabilitate	Rezultatele sunt controlate și verificate manual.	Necesită verificare, deoarece pot apărea omisiuni sau presupuneri incorecte.

În urma comparației, observăm că testele realizate manual sunt mai riguroase din punct de vedere metodologic, deoarece fiecare caz de test este derivat dintr-o tehnică precisă. Acestea oferă o acoperire clar justificată a domeniului de intrare, a frontierelor, a ramurilor de execuție și a mutațiilor posibile.

Testele generate de AI sunt utile în special ca punct de plecare, deoarece pot produce rapid exemple relevante și pot sugera scenarii care ulterior sunt rafinate manual. Totuși, acestea nu înlocuiesc analiza realizată de tester, deoarece nu garantează automat acoperirea completă a tuturor tehnicilor cerute.

Prin urmare, AI-ul a fost folosit ca instrument auxiliar, nu ca înlocuitor al procesului de testare. Contribuția principală a echipei a constat în selectarea, verificarea, corectarea și organizarea testelor conform metodelor teoretice studiate.

5.9 Concluzie

Prin utilizarea inteligenței artificiale, am obținut rapid o clasă completă de teste unitare pentru clasa `EquationSolver`. Promptul formulat a fost detaliat și a inclus atât cerințele funcționale, cât și tehnicile de testare dorite. Output-ul AI a generat teste pentru acoperirea instrucțiunilor, ramurilor, condițiilor, valorilor de frontieră, claselor de echivalență, categoriilor de intrări, căilor de execuție și mutațiilor.

Această abordare demonstrează faptul că instrumentele AI pot fi utile în procesul de testare software, mai ales atunci când promptul este clar, complet și conține reguli precise. Rezultatul obținut poate fi folosit ca bază pentru testarea automată a clasei `EquationSolver` și poate fi extins ulterior în funcție de cerințele proiectului.

Capitolul 6

Concluzii

6.1 Sinteza tehnicilor de testare utilizate

Tehnică	Rol în proiect
Partiționare în clase de echivalență	Identificarea claselor valide și invalide de intrare.
Analiza valorilor de frontieră	Testarea cazurilor-limită pentru $a = 0$ și $\Delta = 0$.
Partiționare în categorii	Organizarea sistematică a combinațiilor de test și reducerea cazurilor redundante prin constrângeri.
CFG	Modelarea fluxului de control al metodei <code>solve()</code> și evidențierea ramurilor de execuție.
Coverage.py	Validarea acoperirii de 100% a codului prin rularea automată a testelor.
Complexitate ciclomatică	Determinarea numărului de drumuri independente necesare pentru acoperirea completă a metodei.
Mutation testing	Evaluarea calității suitei de teste prin introducerea unor modificări controlate în cod.
Cosmic Ray	Generarea automată a mutanților și calculul scorului de mutație.

6.2 Limitări și direcții viitoare

Deși suita de teste obține o acoperire de 100% la nivel de instrucțiuni și un scor de mutație ridicat, există câteva direcții prin care testarea poate fi extinsă. În primul rând, pot fi adăugate mai multe cazuri cu valori reale foarte mari sau foarte mici, pentru a observa comportamentul programului în situații de precizie numerică limitată.

În al doilea rând, se poate utiliza testarea aleatoare sau fuzz testing pentru generarea automată a unor combinații variate de coeficienți a , b și c . Acest lucru ar putea evidenția cazuri particulare care nu au fost incluse manual în suita de teste.

În plus, mutanții supraviețuitori identificați de Cosmic Ray pot fi analizați individual, pentru a decide dacă reprezintă mutanți echivalenți sau dacă necesită adăugarea unor teste suplimentare.

Bibliografie

- [1] Universitatea din București, Facultatea de Matematică și Informatică, *Testarea Sistemelor Software – suport de curs*, materiale de curs, an universitar 2025–2026.
- [2] Paul Ammann, Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2nd Edition, 2016. Disponibil la: <https://www.cambridge.org/highereducation/books/introduction-to-software-testing/95E57CCADEA697EC8594F03729F47311>
- [3] Python Software Foundation, *unittest — Unit testing framework*, Python Documentation. Disponibil la: <https://docs.python.org/3/library/unittest.html>
- [4] Ned Batchelder, *Coverage.py Documentation*. Disponibil la: <https://coverage.readthedocs.io/>
- [5] Graphviz, *Graphviz Documentation*. Disponibil la: <https://graphviz.org/documentation/>
- [6] ISTQB, *Equivalence Partitioning — ISTQB Glossary*. Disponibil la: https://glossary.istqb.org/en_US/term/equivalence-partitioning
- [7] ISTQB, *Boundary Value Analysis — ISTQB Glossary*. Disponibil la: https://glossary.istqb.org/en_US/term/boundary-value-analysis
- [8] Thomas J. McCabe, *A Complexity Measure*, IEEE Transactions on Software Engineering, Vol. SE-2, No. 4, 1976. Disponibil la: <https://ieeexplore.ieee.org/document/1702388>
- [9] Yue Jia, Mark Harman, *An Analysis and Survey of the Development of Mutation Testing*, IEEE Transactions on Software Engineering, Vol. 37, No. 5, 2011. Disponibil la: <https://web.eecs.umich.edu/~weimerw/2022-481F/readings/mutation-testing.pdf>
- [10] OpenAI, *ChatGPT*. Disponibil la: <https://chatgpt.com/>